K. J. Somaiya School of Engineering
Department of Computer Engineering

Project B.H.A.R.A.T: Bringing Historical Archives to Recognition and Translation

Submitted In Partial Fulfillment of Requirements
For the Degree Of

Bachelor of Technology
(Computer Engineering)

By

Aryan Gokul Raut
Roll No: 16010123080
Gaurav Matolia
Roll No: 16010123121
Tanmay Chinchore
Roll No: 160010123135

Guide

Dr. Vaibhav P. Vasani

Somaiya Vidyavihar University

K. J. Somaiya School of Engineering

Certificate

This is to certify that the dissertation report entitled "**Project B.H.A.R.A.T: Bringing Historical Archives to Recognition and Translation**" submitted by the following students at the end of semester VI of TY B. Tech is a bona fide record for partial fulfillment of requirements for the degree Bachelor of Technology (Computer Engineering) of Somaiya Vidyavihar University

16010123080 - Aryan Gokul Raut
16010123121 - Gaurav Matolia
16010123135 -Tanmay Harshad Chinchore

19/05/2026

Guide **Dr. Vaibhav P. Vasani**

(Dr. Deepak. H. Sharma)

Head of the Department

Principal

Date: 19/05/2026
Place: Mumbai-77

Somaiya Vidyavihar University

K. J. Somaiya College of Engineering

DECLARATION

We declare that this written report submission represents the work done based on our and / or others' ideas with adequately cited and referenced the original source. We also declare that we have adhered to all principles of intellectual property, academic honesty and integrity as we have not misinterpreted or fabricated or falsified any idea/data/fact/source/original work/ matter in my submission.We understand that any violation of the above will be cause for disciplinary action by the college and may evoke the penal action from the sources which have not been properly cited or from whom proper permission is not sought.

Signature of the Student 16010123080 **Roll No.**	**Signature of the Student** 16010123121 **Roll No.**
Signature of the Student 16010123135 **Roll No.**	

Date: 19/05/2026

Place: Mumbai-77

Somaiya Vidyavihar University

K. J. Somaiya College of Engineering

Certificate of Approval of Examiners

We certify that this dissertation report entitled **Project B.H.A.R.A.T: Bringing Historical Archives to Recognition and Translation** submitted by **Aryan Gokul Raut, Gaurav Matolia, Tanmay Harshad Chinchore** at the end of semester VI of TY B. Tech is a bona fide record for partial fulfillment of requirements for the degree Bachelor of Technology (Computer Engineering) of Somaiya Vidyavihar University

19/05/2026
Dr Vaibhav P. Vasani

Internal Examiner

N. Singh

External Examiner

Date: 19/05/2026

Place: Mumbai-77

Abstract

Project B.H.A.R.A.T (Bringing Historical Archives to Recognition and Translation) focuses on the recognition and transliteration of historical Modi script (मोडी लिपि) documents into Marathi Devanagari using Machine Learning and Deep Learning techniques. Modi script (मोडी लिपि) was widely used for administrative and historical documentation during the Maratha period and holds significant cultural and historical importance. However, due to the decline in its usage and the limited number of experts capable of reading Modi manuscripts, a large collection of historical records remains inaccessible in digital form.

Recognition of Modi script is a challenging task because of its cursive writing style, absence of clear word boundaries, similarity among character structures, and degradation present in historical manuscripts. In addition, the lack of large standardized datasets further increases the complexity of developing reliable recognition systems.

The proposed system aims to develop an end-to-end pipeline capable of processing raw historical Modi documents and generating Marathi Devanagari transliterations. The methodology includes preprocessing, line segmentation, character segmentation, character recognition, transliteration, character merging, and context-based word separation.

As part of the current phase of the project, an extensive literature survey of approximately twenty research papers related to Modi OCR, segmentation, transliteration, and deep learning-based recognition systems has been completed. Multiple research models including ResNet152, InceptionResNetV2, and Modified CNN architectures have been reimplemented using different preprocessing approaches such as Otsu and ToZero thresholding for comparative analysis.

The project also involves the creation of a custom dataset consisting of Modi vowels, consonants, joint characters, handwritten samples, historical manuscript images, and a synthetic dataset containing approximately 400,000 generated character images. At present, preprocessing, line segmentation, and character segmentation modules have been successfully implemented. Future work will focus on complete character recognition, transliteration into Marathi Devanagari, and optimization of the overall historical document processing pipeline.

***Key words*:** Modi Script Recognition, OCR, Marathi Devanagari Transliteration, Deep Learning, Character Segmentation, Historical Manuscript Processing, CNN, Image Preprocessing, Synthetic Dataset, Historical Document Digitization

Contents

List of Figures..... 9

List of Tables..... 10

Nomenclature..... 11

1 Introduction..... 13

1.1 Background of Modi Script..... 13

1.2 Historical Importance..... 13

1.3 Problem Statement 14

1.4 Objectives..... 14

1.5 Scope of the Project..... 15

1.6 Proposed System Overview(Pipeline)..... 15

2 Literature Review..... 17

2.1 Introduction..... 17

2.2 Traditional OCR Approaches - SVM,KNN..... 17

2.2.1 Support Vector Machines..... 17

2.2.2 K-Nearest Neighbors..... 18

2.3 Deep Learning Based Recognition - CNN, ResNet..... 19

2.3.1 CNN..... 19

2.3.2 ResNet..... 19

2.3.3 Transfer Learning and Hybrid Deep Architecture..... 20

2.4 Transliteration Models - LSTM, CRNN..... 20

2.4.1 Long Short Term Memory(LSTM) Networks..... 20

2.4.2 Convolution Recurrent Neural Networks(C-RNN)..... 21

2.4.3 Vision Language Models and Transformers..... 21

2.5 Dataset Creation Approaches…….. 22

2.6 Segmentation Techniques……. 23

2.6.1 Image Preprocessing and Binarization……. 23

2.6.2 Projection Profiles and Connected Components……. 23

2.7 Comparative Analysis Table……. 24

3 System Methodology……. 27

3.1 Overall Pipeline Architecture……. 27

3.2 Phase 1: Image Processing……. 28

3.2.1 Stage1: Grayscale Conversion ……. 29

3.2.2 Stage2: Noise Removal via Bilateral Filtering……. 29

3.2.3 Stage3: Adaptive Gaussian Thresholding(Binarization)…… 30

3.2.4 Stage4: Speckle Removal via Connected Component Analysis……. 31

3.2.5 Stage5: Projection Profile Variance Deskewing……. 33

3.2.6 Summary of Preprocessing Contributions……. 34

3.3 Phase 2: Line-Level Segmentation……. 35

3.3.1 Anisotropic Gaussian Blob Segmentation……. 36

3.3.2 Autocorrelation Based Frequency Analysis……. 38

3.3.3 Iterative Masking Loop…… 40

3.4 Software Architecture and Development GUI…… 41

3.4.1 Module Organisation…… 42

3.4.2 Key Libraries…… 42

3.4.3 GUI Design and Key Features…… 43

4 Dataset Creation and Preparation…… 45

4.1 Need for Dataset Creation…… 45

4.2 Challenges in Existing Datasets…… 46

4.3 Historical Manuscript Images…… 47

4.4 Character Dataset Preparation (Barakhadi)…… 48
4.5 Handwritten Character Collection…… 51
4.6 Synthetic Dataset Generation…… 52
5 Model Reimplementation and Experiments…… 54
5.1 Selected Research Papers…… 54
5.2 Reimplemented Models…… 54
5.2.1 Custom Shallow Topography: Modified CNN…… 54
5.2.2 Transfer Learning Paradigms: Dropout Induced Incremental Framework 56
5.3 Experimental Environment & Training Configuration…… 59
5.3.1 Dataset Profile…… 59
5.3.2 Hardware and Software Environment…… 59
5.3.3 Core Hyperparameter Scaling Matrix…… 60
5.4 Results and Analysis…… 61
5.4.1 Modified CNN…… 61
5.4.2 InceptionResNetV2 + Otsu…… 62
5.4.3 InceptionResNetV2 + ToZero…… 63
5.4.4 ResNet152 variants…… 64
5.4.5 Key Operational Deviations…… 66
5.4.6 Performance Metrics Summary…… 67
6 Future Scope…… 68
7 Conclusion…… 69
References…… 70
Acknowledgements…… 72

List of Figures

1.1	Modi ShatVidhi Architecture: Unique 6 step process for Modi to Devanagari Transliteration	16
3.1	Preprocessing pipeline for archaic Modi script documents. Each box represents a processing stage with its key parameters.	28
3.2	Line-level segmentation pipeline showing the Anisotropic Gaussian Blob engine and the iterative ACF-driven parameter tuning loop.	35
3.3	Interaction between Python modules, classes, and functions across the pipeline. Arrows indicate function calls and data flow.	41
4.1	Types of dataset collection for MODI Scrip	45
4.2	Historical Documents in Modi Script	47
4.3	Handwritten Documents in Modi Script after 19th century	52
5.1	Reimplementation Strategy for Modified CNN	55
5.2	Reimplementation Strategy for Dropout Approach	57
5.3	Model Accuracy and Loss Graph for Modified CNN Reimplementation Results	61
5.4	Model Accuracy and Loss Graph for InceptionResnetV2+Otsu Reimplementation Results	62
5.5	Model Accuracy and Loss Graph for InceptionResnetV2+Tozero Reimplementation Results	63
5.6	Model Accuracy and Loss Graph for Resnet152+Otsu Reimplementation Results	64
5.7	Model Accuracy and Loss Graph for Resnet152+Tozero Reimplementation Results	65

List of Tables

2.1	Analysis of Existing methodologies	23
3.1	Bilateral Filter Parameters	27
3.2	Adaptive Thresholding Parameters	29
3.3	Speckle Removal Parameters	31
3.4	Deskewing Parameters	32
3.5	Anisotropic Gaussian Segmenter Parameters	36
3.6	ACF-Derived Parameter Mapping	38
3.7	Python Module Overview	41
4.1	Vowels in Modi Script	47
4.2	Consonants in Modi Script	48
4.3	Joint Characters in Modi Script	50
4.4	Collected Dataset info	52
5.1	Reimplementation Dataset description	58
5.2	Hyperparameter Initialization Settings Across Reimplemented Models	59
5.3	Modified CNN Reimplementation Results	60
5.4	InceptionResnetV2+Otsu Reimplementation Results	61
5.5	InceptionResnetV2+ToZero Reimplementation Results	62
5.6	Resnet152+Otsu Reimplementation Results	63
5.7	Resnet152+ToZero Reimplementation Results	64

Nomenclature

B	Bias for hyper-plane in SVM
C	Regularization parameter for SVM
C_k	Category to which documents belong (C_1 or $C_{0)}$
Di	i^{th} Document
$D(x)$	Decision function
FP	False positive
FN	False negative
F1	Evaluation parameter combining precision and recall
IDF_t	Inverse Document Frequency for the term t
K	Number of nearest neighbours(training documents) with which distance computation is to be performed
L	Words present in the document
M_i	Multinomial model feature vector for the i^{th} document in training
M_{it}	Frequency of word W_t in document D_i
M_j	Multinomial model feature vector for the j^{th} document in testing
n_i	Total number of words in D_i
N	Number of documents
N_k	Number of documents of class Ck
NF_t	Number of documents containing term t

TP	True positive
TN	True Negative
TF_i	Frequency of i^{th} word in a document
V	Dictionary
W	Word
$\mathbf{w}$	Weight vector for defining hyper-plane is SVM
X	Input feature vector
Y	Output class of a classifier
Z_{ik}	Parameter to indicate Di has class Ck
α_i	Lagrange multipliers
β_i	Lagrange multipliers used for soft margin SVM
λ	Distance parameter
ξ_i	Slack variable of i^{th} training document violating margin

Chapter 1 - Introduction

1.1 Background of Modi Script

Modi script (मोडी लिपि) is an ancient cursive script that was primarily used for writing the Marathi language in medieval Maharashtra. It served as the official administrative script during the reign of the Marathas and Peshwas for maintaining records related to governance, land revenue, military communication, judicial matters, and official correspondence. Due to its writing speed and continuous flow structure, Modi script became widely adopted for administrative purposes over several centuries.

Unlike Devanagari script, Modi was specifically designed for rapid handwriting and efficient documentation. Characters in Modi script are generally connected and written in a flowing manner without frequent pen lifts, which makes the script highly cursive in nature. Although efficient for writing, these characteristics make automatic recognition and transliteration significantly difficult in modern computational systems.

After the adoption of Devanagari as the standard script for Marathi, the use of Modi script gradually declined. As a result, the number of individuals capable of reading and interpreting Modi manuscripts has reduced considerably. Despite this decline, a large volume of valuable historical documents written in Modi script still exists in libraries, museums, archives, and private collections.

1.2 Historical Importance

Modi script holds immense historical and cultural significance as it preserves valuable information related to the social, political, economic, and administrative history of medieval India. Historical records written in Modi script contain details regarding taxation systems, legal proceedings, military strategies, land ownership, royal correspondence, and various aspects of governance during the Maratha Empire.

Many ancient manuscripts and archival records remain preserved in historical institutions and personal collections. However, due to the complexity of the script and the limited availability of experts who can interpret it, a large portion of these documents remains inaccessible to researchers and the general public.

The digitization and transliteration of Modi manuscripts into Devanagari can contribute significantly toward preserving cultural heritage and improving accessibility to historical knowledge. Developing automated systems for recognition and transliteration can also assist historians, linguists, and researchers in studying historical records more efficiently.

1.3 Problem Statement

The recognition and transliteration of historical Modi script documents present several challenges due to the cursive and complex nature of the script. Modi characters are often connected continuously without proper spacing between words, making segmentation difficult. Additionally, several characters possess highly similar structures, increasing ambiguity during recognition.

Historical manuscripts also suffer from degradation issues such as faded ink, stains, skewness, uneven illumination, and document damage, which further affect recognition accuracy. Another major challenge is the lack of standardized and large-scale annotated datasets for training deep learning models.

The primary objective of this project is to develop a Machine Learning and Deep Learning based framework capable of processing raw historical Modi manuscripts and generating Marathi Devanagari transliterations through multiple stages of document analysis and recognition.

1.4 Objectives

The major objectives of Project B.H.A.R.A.T are as follows:

- To conduct a comprehensive literature review and baseline analysis of existing research work related to Modi Script character recognition, word-level recognition, OCR systems, and transliteration methodologies in order to identify current architectural and dataset limitations.
- To design and implement a standardized preprocessing pipeline architecture incorporating advanced image preprocessing techniques such as binarization, deskewing, thresholding, and noise removal for improving the quality and readability of historical Modi manuscripts.
- To develop a curated hand-drawn and historical dataset by compiling manually written Modi character samples along with authentic historical manuscript images collected from archival and personal sources, while minimizing dependency on synthetic font-based datasets.
- To design and develop an image processing based character recognition model along with an Artificial Intelligence based word-level extraction model for efficient recognition and transliteration of Modi script documents.
- To train, evaluate, and benchmark the performance of the proposed system for character-level classification and recognition using different Machine Learning and Deep Learning approaches.

1.5 Scope of the Project

The scope of this project includes the development of a recognition and transliteration framework for historical Modi script documents using Machine Learning and Deep Learning techniques. The project focuses on preprocessing of historical manuscripts, segmentation of lines and characters, recognition of Modi characters, and transliteration into Marathi Devanagari.

The work also includes the preparation of custom datasets using handwritten samples, historical manuscripts, and synthetic character generation techniques. Multiple deep learning architectures are studied and reimplemented to analyze their effectiveness in recognizing Modi characters.

Although the current phase of the project mainly focuses on preprocessing, segmentation, dataset preparation, and model analysis, the final objective is to develop a complete transliteration system capable of processing full historical documents.

1.6 Proposed System Overview (Pipeline)

The proposed system is designed as a multi-stage pipeline for the recognition and transliteration of historical Modi manuscripts into Marathi Devanagari. We are proposing Modi ShatVidhi: A unique 6 step process that transliterates an input archaic document written in Modi Script to Devanagari Script. The pipeline focuses on processing raw archival documents through a sequence of image processing, segmentation, recognition, and contextual reconstruction stages in order to generate machine-readable transliterated text.

The workflow begins with raw archaic Modi documents collected from historical archives and handwritten sources. Since historical manuscripts often contain degradation such as faded ink, skewness, stains, uneven illumination, and background noise, the first stage of the pipeline involves preprocessing. This stage includes grayscale conversion, thresholding, binarization, deskewing, contrast enhancement, and noise removal techniques to improve document clarity and prepare the images for further analysis.

After preprocessing, the enhanced document undergoes line segmentation, where individual text lines are separated from the complete manuscript image. Accurate line segmentation is important because Modi script is highly cursive and text is generally written in a continuous flowing manner.

The segmented lines are then passed to the character segmentation stage. In this stage, individual Modi characters are extracted from the text lines using image processing techniques. Character segmentation is one of the most challenging stages due to the connected writing style of Modi script and the absence of proper spacing between characters and words.

Following segmentation, the isolated characters are processed through deep learning based recognition models trained on curated Modi datasets. The recognition stage identifies the extracted characters and maps them to their corresponding Marathi Devanagari representations.

Multiple architectures such as ResNet152, InceptionResNetV2, and Modified CNN models are studied and evaluated for recognition performance.

Once character-level recognition is completed, the recognized characters are merged to reconstruct words and meaningful textual sequences. Since Modi manuscripts generally do not contain clear word boundaries, the pipeline further incorporates a context-based word separation mechanism to identify probable word formations and improve transliteration readability.

The final stage of the system generates Marathi Devanagari transliterations of the input Modi manuscript in machine-readable form. The proposed pipeline aims to create a structured and scalable framework capable of assisting in the digitization, preservation, and accessibility of historical Modi documents.

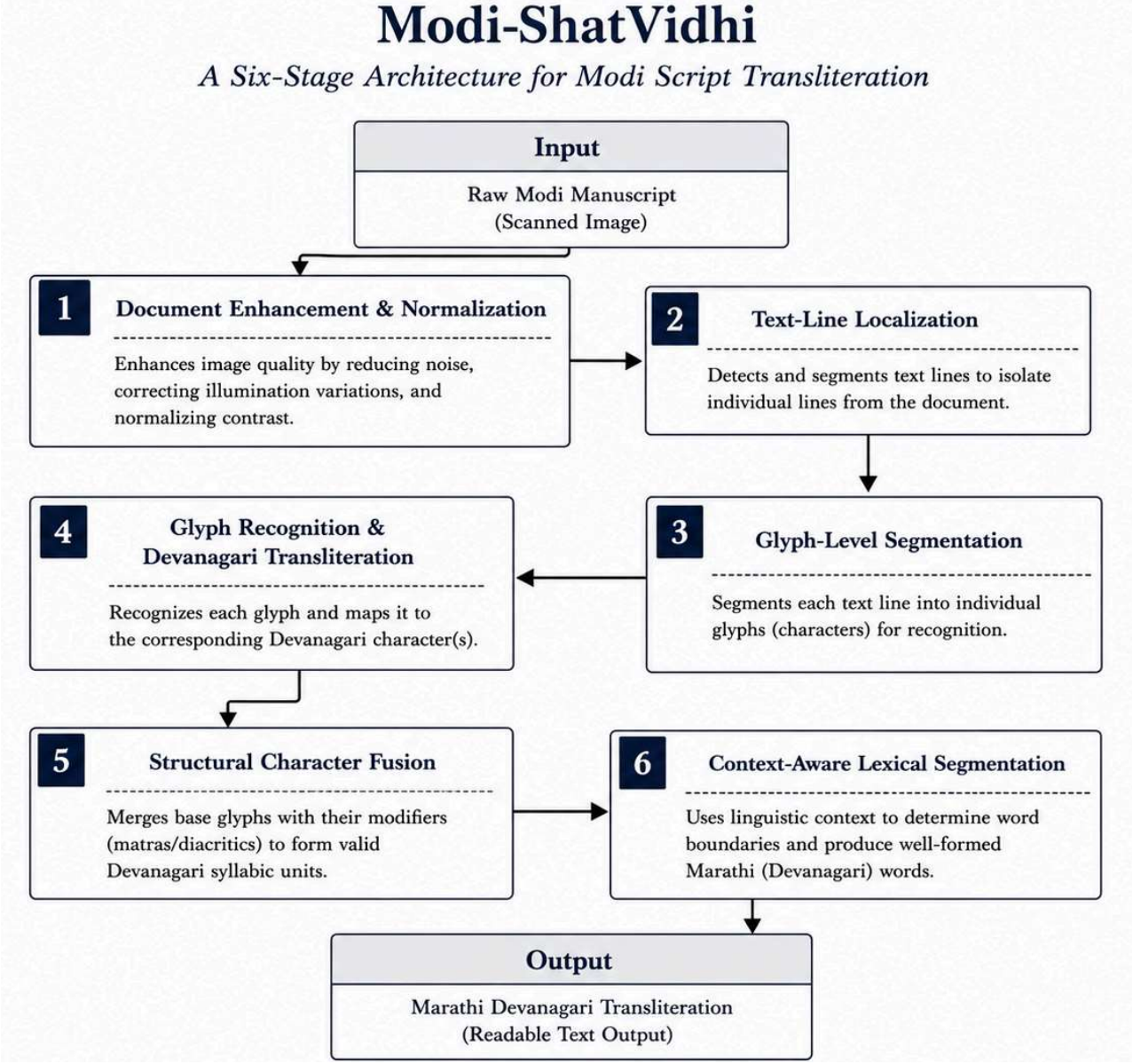


Fig 1.1: Modi ShatVidhi Architecture: Unique 6 step process for Modi to Devanagari Transliteration

Chapter 2 - Literature Review

2.1 Introduction

The preservation of cultural heritage through digital archiving has emerged as a crucial intersection of historical linguistics and computer science. The Modi script—an ancient cursive writing system belonging to the Indo-Aryan language family—was extensively used between the 12th and mid-20th centuries for administrative, official, and legal documentation in the Maratha Empire across Maharashtra, Gujarat, Karnataka, and central India. Scribes traditionally utilized a boru (bamboo pen) to write fluidly beneath a continuous horizontal baseline without lifting the pen. This creates highly connected, stylized, and cursive structural forms with intense variations in handwriting style.

Because the modern user base of the script has declined to a minute population of linguists and historians, developing automated Optical Character Recognition (OCR) systems is crucial to unlock thousands of official and historical "Shivakalin" and "Peshvekalin" documents. However, complex overlapping strokes, visual similarities among characters, data scarcity, and manuscript degradation make historical Modi text recognition a highly challenging task for conventional digitization systems.

2.2 Traditional OCR Approaches - SVM, KNN

Early and resource-constrained attempts at automating Modi script recognition focused heavily on traditional machine learning frameworks. These methodologies rely on well-defined statistical and mathematical principles and explicit feature engineering pipelines where handcrafted structural, textural, or geometric features are extracted prior to classification.

2.2.1 Support Vector Machines (SVM)

Support Vector Machines have consistently established themselves as the most robust traditional classifiers due to their ability to maximize classification margins in high-dimensional feature spaces.

Early Baselines: Kumar et al. (2017) utilized an SVM classifier combined with binarization and morphological preprocessing on clean documents, achieving an accuracy of 78%. However, its real-world application was heavily restricted since the system could not handle noisy data or damaged manuscripts.

Feature-Engineered Frameworks: Deshmukh and Kolhe (2023) introduced a system leveraging Gradient, Structural, and Concavity (GSC) feature extraction alongside a Minimum Distance/SVM-adjacent framework. By automating feature vector binarization, their proposed system secured an 83.19% identification rate across 55 distinct classes of 95,000 historical handwritten characters.

Empirical Comparison: In a rigorous comparative study using a dataset of 10,110 scanned character images, Bhong and Gupta (2025) demonstrated that SVM achieved a peak traditional classification accuracy of 96.4%, with a precision of 0.97, a recall of 0.95, and an F1-score of 0.96, validating its superiority over other classical machine learning algorithms.

2.2.2 K-Nearest Neighbors (KNN) & Other Classical Classifiers

Instance-based, probabilistic, and ensemble classifiers have been explored as lightweight, interpretable, and computationally efficient alternatives.

K-Nearest Neighbors (KNN): KNN models classify characters based on distance metrics within the engineered feature space. Bhong and Gupta (2025) reported a baseline performance of 92.2% accuracy for KNN with a precision of 0.93 and F1-score of 0.92. However, system benchmarks show that when evaluated on unconstrained historical scripts with extensive stroke noise, standalone KNN classification rates drop to approximately 75%.

Decision Trees (DT) and Random Forest (RF): Rule-based hierarchical split models demonstrate strong capability in handling complex feature interactions. A standard Decision Tree achieves a modest accuracy of 91.3%. However, it suffers from overfitting when handwriting styles change drastically. Ensemble modeling via Random Forest rectifies this deficiency by aggregating multiple randomized decision trees. This raises the classification accuracy to 92.1% (with an F1-score of 0.92) and shows notable robustness against noisy data inputs.

Naive Bayes (NB): Operating on probabilistic independence, Naive Bayes models execute at high computational speeds with low training times. They stabilize around 92.1% accuracy for tightly bound character datasets but struggle with high-dimensional variations in script structures.

Ultimately, while traditional machine learning systems are highly efficient in low-resource settings, their real-world application remains restricted; they rely heavily on pristine input files and struggle to adapt to the random stroke variations of real handwriting.

2.3 Deep Learning Based Recognition - CNN, ResNet

To bypass the brittle nature of handcrafted feature extraction, contemporary literature focuses sharply on deep learning models that automatically learn spatial hierarchies, fine-grained visual patterns, and stroke intersections directly from text images.

2.3.1 Convolutional Neural Networks (CNN)

CNN architectures utilize shared weights and pooling layers to maintain spatial invariance, which has redefined the benchmarks for offline Modi character recognition.

Early Adoptions: Sharma et al. (2018) pioneered standard CNN applications in this domain, achieving an accuracy of 85% by automatically isolating intricate spatial structures within the characters.

Model Configurations: Continued architecture tuning has yielded massive performance gains. Implementing an AlexNet architecture in MATLAB over 58 distinct Modi characters with 100 graphic variations each yielded a success rate of 89.72%.

Custom OCR Engines: Advanced Convolutional networks specifically tailored into a Modi Script Character Recognizer System (MSCR) have successfully classified all 48 core characters (vowels, consonants, and 10 numerals). The training progress of these custom CNN engines yields an accuracy of 73.93% to 76.46% over extensive unconstrained regional variations, utilizing dense softmax cross-entropy heads for final probability assignment. Standing benchmarks show standalone custom CNNs reaching up to 96% accuracy on clean verification data.

2.3.2 Residual Networks (ResNet) & Advanced Inception Models

Deep networks are traditionally susceptible to the vanishing gradient problem when extracting fine-grained patterns from high-density script frames. Residual Networks overcome this obstacle using shortcut/skip connections.

ResNet Implementations: Rathod et al. (2019) leveraged a ResNet50 framework to access deeper spatial abstractions without performance degradation, boosting offline recognition accuracy to 89%–94.55% with a model precision of 0.86. The current peak state-of-the-art for character-level tracking has been established by coupling a highly optimized ResNet-9 framework, hitting a maximum test accuracy of 98.92%.

Inception Networks: Cross-evaluations with Inception V3 networks—which extract multi-scale text features in parallel through varied kernel sizes—yielded a testing accuracy of 93.92% with a precision of 0.843.

2.3.3 Transfer Learning and Hybrid Deep Architectures

When dealing with historical text, data scarcity is a persistent bottleneck. Transfer learning addresses this issue by adapting models pre-trained on massive open-source datasets (such as ImageNet) or related scripts (like Devanagari or Sanskrit) to the Modi domain.

VGG & Domain Adaptation: Verma et al. (2018) fine-tuned a pre-trained VGG16 network on minimal Modi training points, drastically slashing training times while securing an 88% accuracy benchmark. Modern iterations of VGG16 have pushed these training metrics up to 97.64% with a 91% validation holdout. Similarly, Rajput et al. demonstrated that adapting pre-trained Devanagari engines to Modi via cross-script learning enhanced overall accuracy by 8%.

Hybrid Frameworks: Fusing feature extraction with robust boundary classifiers has generated elite hybrid structures. A CNN-SVM approach, where CNNs extract features and SVMs do classification, leverages deep spatial features alongside maximum-margin decision boundaries. This hybrid architecture improves single-model CNN tracking by up to 10%, reaching peak structural classification accuracies of 99.3% when utilizing highly localized deep maps.

Attention Mechanisms: Integrating localized Attention Mechanisms directly into CNN and ResNet cores forces the network to dynamically shift computational focus onto highly distinct cursive character strokes and overlapping lines. This drives standard historical document tracking efficiencies past 92% – 93%.

2.4 Transliteration Models - LSTM, CRNN

Character-level tracking fails when confronted with fluid, continuous words where characters are structurally bound by horizontal baselines. Resolving word-level lines requires sequence-to-sequence models that track temporal dependencies and stroke flow transitions.

2.4.1 Long Short-Term Memory (LSTM) Networks

Because the cursive structure of Modi script introduces distinct sequential and directional dependencies within word formations, sequential architectures are critical.

Sequence Modeling: Standalone LSTM transcription frameworks applied directly to complete historical lines demonstrate strong sequence alignment, reaching word character transcription accuracies of 94.67%.

Spatial-Temporal Fusions: Fusing spatial image processing with temporal memory arrays has produced highly successful hybrid configurations. Deshmukh et al. combined CNN feature maps with standard LSTM arrays to map spatial visual data alongside sequential character logic, boosting recognition results by 12% over baseline CNN models without losing spatial-temporal text continuity.

2.4.2 Convolutional Recurrent Neural Networks (C-RNN)

A standard C-RNN pipeline combines an upfront CNN architecture for frame-by-frame visual feature extraction, followed by recurrent layers (such as bidirectional LSTMs) to interpret character sequence states, finalized by Connectionist Temporal Classification (CTC) loss layers for character alignment.

BiLSTM Tracking: Rajesh et al. (2023) validated a hybrid CNN-BiLSTM sequential architecture, capturing spatial layouts via the CNN component while evaluating multi-directional sequence context via the Bidirectional LSTM layer, returning a 91% continuous recognition accuracy on cursive sequences.

Pipeline Applications: Implementations of custom C-RNN software (such as the Calamari OCR framework) for automated handwritten Modi-to-Devanagari transliteration have achieved an 88.14% character-level sequence mapping, though full-word matching drops to 61% when navigating unsegmented document distortions.

2.4.3 Vision-Language Models (VLMs) and Transformers

The frontier of word-level document transcription relies on multi-layered self-attention networks.

Self-Attention Systems: Bhagat et al. (2023) developed a self-attention transformer model to trace explicit inter-stroke context across word strings, surpassing all standard CNN-LSTM configurations to secure elite accuracy profiles.

Knowledge Distillation: To compress large systems for practical use, modern research has turned to Knowledge Distillation. Here, large, highly parameter-heavy Vision-Language Models (VLMs) or Generative LLMs (such as LLaMA-2/3 or Mistral 7B) act as "Teacher" networks to guide compact, efficient "Student" networks.

A prominent example is MoScNet-XL, a 429-million parameter model that distills complex structural rules into a lightweight, deployable student transformer. This system achieves exceptional language representation scores, yielding BLEU evaluation metrics of 51.03 on the MoDeTrans benchmark and 53.97 on synthetic datasets (SynthMoDe).

2.5 Dataset Creation Approaches

A recurring theme across all reviewed literature is the acute shortage of labeled, verified public text repositories for the Modi script, which severely limits model training. Researchers have employed distinct data engineering methodologies to combat data scarcity.

Manual Curation and Historical Sourcing

MODI-HHDoc Dataset: Developed by Manisha Deshmukh and Satish Kolhe (2023), this represents a crucial foundational baseline repository. It contains 3,350 unique historical images sourced directly from authentic administrative archives. It serves as a real-world testing ground for validating digitization, transcription, and transliteration algorithms.
Barakhadi Assemblies: Specialized tracking sets have been manually captured from diverse human authors to ensure a robust representation of handwriting styles. A notable example is a multi-author database containing 7,721 isolated character variants written by 25 distinct individuals. Similarly, alternative evaluation sets contain 10,110 targeted character instances acquired via open platforms (e.g., Kaggle) to analyze specific script variations.

Synthetic Augmentation Pipelines

To prevent overfitting in deep neural networks due to limited sample sizes, researchers deploy advanced on-the-fly and synthetic transformation pipelines.

Geometric & Affine Generators: Standard Keras ImageDataGenerator pipelines apply random scaling, shifts, and angular rotations ranging from -30° to +30° to artificially expand training arrays.
Stroke Distortion Operations: Specialized custom data augmentation methods introduce variations in stroke thickness, character orientation, and size. This process expands training diversity and yields an approximate 8% improvement in generalization accuracy.

Generative Adversarial Networks (GANs)

The modern paradigm for synthesizing realistic data utilizes adversarial networks to model human handwriting variations and document degradation.

Synthetic Generation: Kulkarni et al. (2021) trained GANs on limited original characters to output novel, high-fidelity synthetic character images, which raised standard CNN performance by 10%.
Attention-Augmented GANs: Pathak et al. (2021) and Ghosh et al. (2023) advanced this technique by introducing Attention-Augmented GANs. These networks inject realistic handwriting inconsistencies, ink bleeding, fading, and historical aging artifacts into generated images. This process allows deep networks to train effectively on diverse, highly realistic synthetic data.

2.6 Segmentation Techniques

Document segmentation transforms raw, continuous manuscripts into structured units suitable for character or word-level classifiers. Because Modi characters are highly interconnected by a continuous baseline, accurate segmentation is a major challenge.

2.6.1 Image Preprocessing and Binarization

Before applying segmentation boundaries, documents must undergo quality enhancement to correct aging artifacts and background noise.

Noise Filtering: Bilateral Filtering (typically utilizing a kernel size of 3) is applied to remove background noise while preserving sharp character edges. Non-linear Median Filtering is also used to target salt-and-pepper artifacts.
Deskewing: This process calculates text base vectors to correct angular misalignments introduced during document scanning.
Binarization: Converting grayscale input into binary profiles is critical. Otsu's Binarization establishes an optimal global threshold, which is highly effective for isolating clean, high-contrast text. However, for degraded manuscripts with uneven illumination or ink stains, researchers implement adaptive local thresholding like Sauvola Binarization or Gaussian Adaptive Thresholding to extract readable character outlines.

2.6.2 Projection Profiles and Connected Components

Vertical Projection Profile (VPP): The foundational method for line and word segmentation relies on projection histograms. VPP maps column-wise pixel densities to locate white-space valleys between characters. However, standard VPP often fails when characters overlap or touch due to the continuous baseline.

Dual-Thresholding VPP Extensions: Tamhankar et al. (2020) resolved overlapping character errors by introducing a modified VPP framework using dual-threshold criteria. This approach refines boundary detection, significantly reducing character segmentation errors in offline cursive text.
Character Isolation Models: Modern pipelines introduce explicit character isolation models before the classification stage. Pre-processing words to segment individual characters balances visual complexity, preventing surrounding cursive strokes from distorting downstream classification accuracy.

2.7 Comparative Analysis Table

Table 2.1: Analysis of Existing methodologies

Classification Tier	Model	Feature Extraction Approach	Key Dataset Utilized	Performance Metrics	Key Advantages	Operational Limitations
Traditional Machine Learning	Support Vector Machine (SVM)	Handcrafted Features (GSC, Edges, Contours)	10,110 images (Manisha Deshmukh collection)	96.4% Classification Accuracy	Low computational cost; highly effective in high-dimensional spaces; interpretable.	Dependent on clean data inputs; poor generalization to raw handwriting variations.
Traditional Machine Learning	k-Nearest Neighbors (k-NN)	Distance Vector Mapping	Isolated Scanned Characters Dataset	92.2% (Controlled Benchmark) / ~75% (Historical Text)	Simple setup; highly efficient for small, tightly bounded tracking sets.	High memory usage; accuracy drops significantly on noisy or cursive text.

Traditional Machine Learning	Random Forest (RF)	Decision Tree Ensemble Splits	Isolated Scanned Characters Dataset	92.1% Accuracy	Robust against noisy inputs; handles complex feature interactions well.	Complex structure; generally falls below SVM performance limits.
Deep Learning (Character-Level)	Convolutional Neural Network (CNN)	Automated Spatial Hierarchies	Custom Modi Barakhadi Arrays	96.0% Classification Accuracy	Eliminates manual feature engineering; highly accurate stroke pattern tracking.	Requires large training sets; high computational resource demands.
Deep Learning (Character-Level)	ResNet-9 / ResNet-50	Skip Connections & Deep Spatial Layering	Real-world Multi-Author Database	98.92% (ResNet-9 Test Accuracy Peak)	Eliminates vanishing gradient issues; extracts ultra-fine stroke characteristics.	High computational footprint; complex architecture configuration.
Deep Learning (Character-Level)	VGG16 (Transfer Learning)	Pre-trained ImageNet Weight Tuning	Limited Modi Training Samples	97.64% Train / 91.0% Val Accuracy	Accelerates training; high precision even with	Limited by the resolution similarity between the source

					restricted target datasets.	and target domains.
Deep Learning (Character-Level)	Hybrid CNN-SVM Architecture	CNN Spatial Maps + Max-Margin Boundaries	Unconstrained Document Arrays	99.3% Structural Accuracy	Combines deep feature extraction with robust margin-based classifiers.	Computationally heavy during training over massive text repositories.
Deep Learning (Word-Level Sequences)	C-RNN (CNN + BiLSTM Hybrid)	CNN Spatial Arrays + Sequence Time-Step Mapping	Continuous Cursive Script Sequences	88.14% Char-level / 61.0% Full Word Matching	Effectively captures chronological layout and flow across continuous words.	Prone to cumulative errors across heavily degraded manuscripts.
Vision-Language / Transformers	MoScNet-XL (Distilled Transformer)	Knowledge Distillation via Generative LLM Teachers	SynthMoDe / MoDeTrans Reference Datasets	BLEU 51.03 (MoDeTrans) / 53.97 (SynthMoDe)	Light, highly compressed model footprint; optimized for real-world deployment.	Heavy dependency on the accuracy of the teacher model's rules.

PART-3: System Methodology

3.1 Overall Pipeline Architecture

This report provides a detailed technical account of the first two completed phases of our end-to-end transliteration pipeline:

Phase 1: Image Preprocessing — A six-stage pipeline that converts a raw archaic scan into a clean, binary, deskewed image suitable for algorithmic consumption.

Phase 2: Line-Level Segmentation — An Anisotropic Gaussian Blob Segmentation engine coupled with an iterative Autocorrelation-based parameter tuning loop that automatically extracts individual text lines from the preprocessed document.

The preprocessing pipeline is grounded in the methodology described by Kausadikar et al. [1] for the creation of the MoDeTrans dataset. We extend their pipeline with two novel additions: (a) an automated connected-component speckle removal stage, and (b) a Projection Profile Variance-based deskewing algorithm that replaces manual Photoshop intervention. The segmentation engine draws from the learning-free Anisotropic Gaussian filter approach of Barakat et al. [2] and augments it with a self-correcting iterative masking loop driven by Autocorrelation Function (ACF) frequency analysis.

3.2 Phase 1: Image Preprocessing

The goal of the preprocessing phase is to transform a raw, noisy, potentially skewed colour scan of an archaic Modi manuscript into a clean binary image with black ink on a white background, perfectly aligned horizontally. The pipeline consists of six sequential stages. Each stage addresses a specific category of degradation.

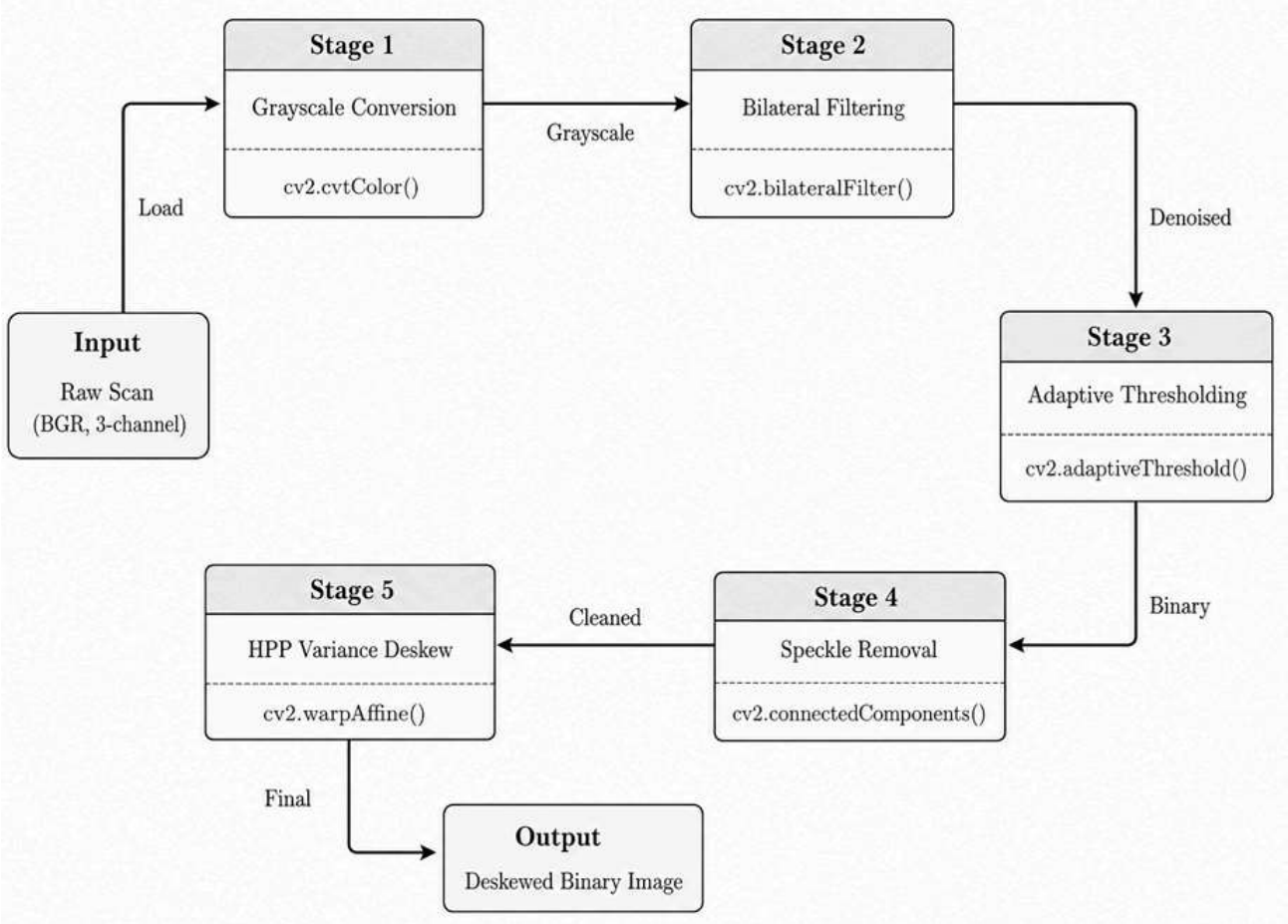


Figure 3.1: End-to-end preprocessing pipeline for archaic Modi script documents. Each box represents a processing stage with its key parameters.

3.2.1 Stage 1: Grayscale Conversion

The input to the pipeline is a raw BGR (Blue-Green-Red) colour image as read by OpenCV. Since all subsequent operations are intensity-based and do not require colour information, the first step is to collapse the three-channel image into a single-channel grayscale representation. This is performed using the standard luminance-weighted formula:

$$I_{gray}(x,\ y)\ =\ 0.299 \cdot R(x,y)\ +\ 0.587 \cdot G(x,y)\ +\ 0.114 \cdot B(x,y)$$

This weighting reflects the human eye's differential sensitivity to the three primary colours, with green contributing the most to perceived brightness. The resulting grayscale image Igray ∈ [0,255] preserves the perceptual contrast between the dark ink strokes and the lighter paper background while reducing the data dimensionality by a factor of three, thereby accelerating all downstream computations.

Implementation Details	
Library	OpenCV(cv2)
API Call	cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
Wrapper	to_grayscale(image_bgr)
Source File	preprocessing/preprocess.py

3.2.2 Stage 2: Noise Removal via Bilateral Filtering

Historical documents are afflicted by high-frequency noise originating from paper grain, scanner artefacts, foxing (brown spots from fungal growth), and micro-tears. A naive Gaussian blur would suppress this noise but would simultaneously erode the sharp edges of the calligraphic strokes—an unacceptable trade-off for a script as intricate as Modi.

We employ the Bilateral Filter [3], an edge-preserving smoothing operator. Unlike a standard Gaussian filter that weights neighbouring pixels solely by their spatial distance from the centre pixel, the bilateral filter introduces a second weighting term based on intensity similarity. The output at pixel (x,y) is defined as:

$$I_{\mathrm{bf}}(x,y) = \frac{1}{W_p} \sum_{(x_i,y_i)\in\Omega} I(x_i,y_i) \cdot \underbrace{e^{-\frac{(x_i-x)^2+(y_i-y)^2}{2\sigma_s^2}}}_{\text{Spatial Gaussian}} \cdot \underbrace{e^{-\frac{(I(x_i,y_i)-I(x,y))^2}{2\sigma_r^2}}}_{\text{Range Gaussian}}$$

where Wp is a normalizing factor, Ω is the neighbourhood of diameter d, σs controls the spatial extent of the filter, and σr controls the sensitivity to intensity differences.
In our implementation, the parameters are configured as follows:

Table 3.1: Bilateral Filter Parameters

Parameter	Symbol	Value
Neighbourhood diameter	d	5
Colour/Range sigma	σ_r $(sigmaColor)$	100
Spatial sigma	σ_s $(sigmaSpace)$	100

The relatively high $\sigma_r = 100$ allows the filter to aggressively smooth noisy paper textures (where neighbouring pixels have similar intensities) while simultaneously preserving the sharp transition at ink-to-paper boundaries (where the intensity difference exceeds the Gaussian falloff). This directly follows the denoising strategy described by Kausadikar et al. [1], who specify bilateral filtering with a small kernel for Modi script preprocessing.

Implementation Details	
Library	OpenCV (cv2)
API Call	cv2.bilateralFilter(gray, d, sigmaColor, sigmaSpace)
Wrapper	remove_noise(gray, cfg: PreprocessConfig)
Source File	preprocessing/preprocess.py
Config Class	PreprocessConfig (Python dataclass)

3.2.3 Stage 3: Adaptive Gaussian Thresholding (Binarization)

The denoised grayscale image must be converted into a binary image where ink pixels are black (0) and paper pixels are white (255). A global threshold (e.g., Otsu's method) fails on archaic manuscripts because the background intensity varies dramatically across the page—the centre of a well-preserved document may have a bright cream background, while the margins may be darkened by centuries of handling, moisture, or foxing.

We employ Adaptive Gaussian Thresholding [1], which computes a local threshold for each pixel based on the weighted mean of its neighbourhood:

$$T(x,y) = \frac{\sum_{(x_i,y_i)\in\mathcal{N}} G_\sigma(x_i - x, y_i - y) \cdot I(x_i, y_i)}{\sum_{(x_i,y_i)\in\mathcal{N}} G_\sigma(x_i - x, y_i - y)} - C$$

where $\mathcal{N}$ is a square neighbourhood of size B × B (the block size), G_σ is a Gaussian weighting kernel, and C is a constant subtracted from the weighted mean to shift the threshold. A pixel is classified as foreground (ink) if $I(x,y) < T(x,y)$, and background otherwise.

Table 3.2: Adaptive Thresholding Parameters

Parameter	Symbol	Default Value
Block size (must be odd)	B	61
Threshold constant	C	21

A large block size of B = 61 pixels was selected to accommodate the wide variation in stroke thickness and local illumination gradients typical of Modi manuscripts. The constant C = 21 provides an aggressive bias toward classifying ambiguous pixels as background, which prevents faint paper textures from being misidentified as ink. An automatic polarity check ensures the output always has dark text on a light background, regardless of the input's original contrast orientation: if the mean pixel value of the binary image falls below 127, the image is inverted.

Implementation Details

Library	OpenCV (cv2)
API Call	cv2.adaptiveThreshold(src, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, blockSize, C)
Wrapper	adaptive_gaussian_threshold(denoised, cfg)
Source File	preprocessing/preprocess.py

3.2.4 Stage 4: Speckle Removal via Connected-Component Analysis

This stage is a novel addition to the pipeline described by Kausadikar et al. [1], who handled residual noise through manual Photoshop-based intervention. We replace this manual step with a fully automated connected-component filtering algorithm.

After binarization, the image typically contains thousands of small isolated white regions (in the inverted foreground) that correspond to paper grain, scanner dust, or microtears—not actual ink strokes. These speckles, if left unchecked, severely degrade the subsequent segmentation stage by creating false peaks in the Horizontal Projection Profile.

Our algorithm operates as follows:

1. **Inversion:** The binary image is inverted so that foreground (ink + noise) pixels become white (255) and background becomes black (0).
2. **Connected-Component Labelling:** OpenCV's 8-connected component analysis (cv2.connectedComponentsWithStats) is applied, which assigns a unique integer label to each connected blob and computes its bounding box dimensions (x,y,w,h) and pixel area A.
3. **Morphometric Filtering:** A component is retained as a legitimate ink stroke if and only if it satisfies at least one of the following conditions:

$$\text{Keep}(c) = \begin{cases} \text{True} & \text{if } A(c) \geq A_{\text{min}} \\ \text{True} & \text{if } h(c) \geq h_{\text{min}} \ \wedge \ w(c) \geq w_{\text{min}} \\ \text{False} & \text{otherwise} \end{cases}$$

4. **Reconstruction:** Only retained components are painted onto a clean canvas, and the result is inverted back to the standard black-text-on-white convention.

Table 3.3: Speckle Removal Parameters

Parameter	Symbol	Value
Minimum component area	A_{min}	100 pixels
Minimum component height	h_{min}	10 pixels
Minimum component width	w_{min}	7 pixels

The dual-criteria approach (area or dimensional thresholds) is critical for Modi script. Many legitimate characters in Modi are narrow vertical strokes that may have small areas but significant height. A purely area-based filter would incorrectly discard these. Conversely, some diacritical marks are short but wide. The disjunctive condition ensures both morphologies are preserved while eliminating the isotropic point-noise that characterizes aged paper.

Implementation Details	
Library	OpenCV (cv2), NumPy (numpy)
API Call	cv2.connectedComponentsWithStats(inv, 8) for component labelling; NumPy array indexing for morphometric filtering and canvas reconstruction
Wrapper	remove_speckle_components(binary, cfg)
Source File	preprocessing/preprocess.py

3.2.5 Stage 5: Projection Profile Variance Deskewing

This stage represents our second novel contribution to the preprocessing pipeline. While Kausadikar et al. [1] mention deskewing as a final step in their preprocessing work flow, no algorithmic details are provided. Our implementation employs the Projection Profile Variance method, a classical and highly robust approach to skew estimation for text-heavy documents [4,5].

The fundamental insight is that when text lines are perfectly horizontal, the Horizontal Projection Profile (HPP)—the row-wise sum of foreground pixels—exhibits the sharpest possible alternation between high-density peaks (text rows) and zero-density valleys (inter-line gaps). Any rotational skew causes these peaks and valleys to blur into each other, reducing the variance of the profile.

Algorithm:

1. **Downscaling:** If the image width exceeds 1000 pixels, it is proportionally down scaled to accelerate the sweep. The angle is computed on the reduced image and applied to the full-resolution original.

2. **Angular Sweep:** TheimageisrotatedthroughN candidateangles $\theta_k \in [-\theta_{max}, +\theta_{max}]$ with step size Δθ:

$$\theta_k = -\theta_{\max} + k \cdot \Delta\theta, \quad k = 0, 1, \ldots, \left\lfloor \frac{2\theta_{\max}}{\Delta\theta} \right\rfloor$$

3. **Variance Computation:** At each angle θk, the HPP hk(y) is computed and its variance is calculated:

$$\mathrm{Var}(\theta_k) = \frac{1}{H} \sum_{y=0}^{H-1} \left(h_k(y) - \bar{h}_k \right)^2$$

4. **Optimal Angle Selection:** The deskew angle is chosen as:

$$\theta^* = \arg\max_{\theta_k} \mathrm{Var}(\theta_k)$$

5. **Rotation:** The full-resolution binary image is rotated by $\theta*$ using affine transformation with cubic interpolation and a white border fill.

Table 3.4: Deskewing Parameters

Parameter	Symbol	Value
Maximum search angle	θ_{max}	$\pm$ 45
Angular step size	$\Delta\theta$	0.5
Total candidate angles	N	41

The $\pm$ 45 range is sufficient for scanned archival documents, which rarely exhibit skew exceeding 5–7. The 0.5 resolution provides sub-degree precision in angle estimation. Together with the downscaling optimization, the entire deskew computation completes in under 1.5 seconds on a standard laptop CPU.

Implementation Details	
Library	OpenCV (cv2), NumPy (numpy)
API Call	np.arange() for angular sweep generation; cv2.getRotationMatrix2D() and cv2.warpAffine() for affine rotation; np.var() for variance computation
Wrapper	estimate_skew_hpp(binary) → returns θ^*; deskew(binary, angle) → returns rotated image
Orchestrator	preprocess_image_array(image_bgr, cfg)
Source File	preprocessing/preprocess.py

3.2.6 Summary of Preprocessing Contributions

To summarise, our preprocessing pipeline faithfully implements the four core stages described by Kausadikar et al. [1]—grayscale conversion, bilateral filtering, adaptive Gaussian thresholding, and deskewing—while introducing two significant enhancements:

1. **Automated Speckle Removal (Section 3.2.4):** Replaces manual Photoshop-based noise removal with a deterministic, reproducible connected-component morphometric filter.
2. **HPP-Variance Deskewing (Section 3.2.5):** Provides a fully automated, algorithmically precise deskewing method where the original paper leaves the implementation unspecified.

3.3 Phase 2: Line-Level Segmentation

Given the clean, deskewed binary image produced by Phase 1, the objective of this phase is to identify the vertical extents (y_{start}, y_{end}) of every individual line of text in the document. These bounding regions are then used to crop and extract each line as a separate image for downstream character-level recognition or VLM-based transliteration.

Our segmentation system comprises two tightly coupled components: (1) the Anisotropic Gaussian Blob Segmenter, a learning-free algorithm inspired by Barakat et al. [2], and (2) an Autocorrelation-based Iterative Parameter Tuning Loop that automati cally derives optimal segmenter parameters from the spatial frequency structure of the document.

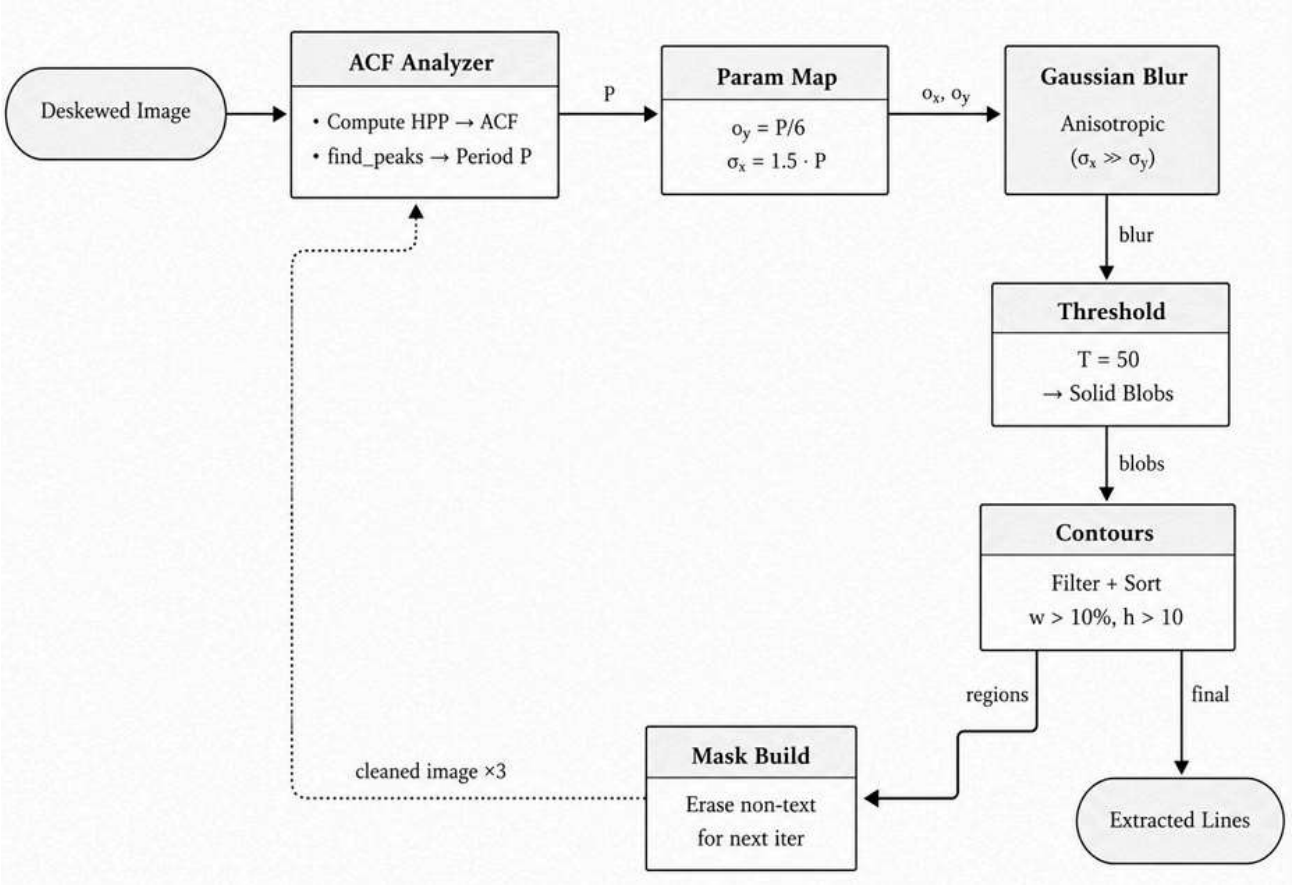


Figure 3.2: Line-level segmentation pipeline showing the Anisotropic Gaussian Blob engine and the iterative ACF-driven parameter tuning loop.

3.3.1 Anisotropic Gaussian Blob Segmentation

The core segmentation algorithm exploits the fundamental geometric property of text documents: characters within a single line are spatially close in the horizontal direction, while adjacent lines are separated by a vertical gap. By applying a directional (anisotropic) Gaussian blur that smears ink strongly in the horizontal direction but only minimally in the vertical direction, individual characters on the same line merge into a single elongated blob, while lines remain separated.

The algorithm proceeds in four stages:

1. **Internal Binarization:** The preprocessed image is re-binarized using adaptive thresholding with a 15 × 15 block and C = 10 to produce an inverted binary mask where ink pixels are white (255).
2. **Directional Gaussian Blur:** An anisotropic Gaussian kernel of size (σ_x , σ_y) is applied to the binary mask. The horizontal kernel width σ_x is set much larger than σ_y to create a strong horizontal smear:

$$I_{\text{smeared}}(x, y) = (I_{\text{bin}} * G)(\sigma_x, \sigma_y)$$

   where $G(\sigma_x , \sigma_y)$ denotes a 2D Gaussian kernel with independent standard deviations along each axis. The key constraint is $\sigma_x \gg \sigma_y$, ensuring horizontal connectivity within a line while preserving vertical separation between lines.

3. **Blob Thresholding:** The smeared image is thresholded at $T_{blob} = 50$ to produce solid white blobs, each representing one text line.
4. **Contour Extraction and Filtering:** External contours of the blobs are detected using OpenCV's findContours. Each contour's axis-aligned bounding box (x, y, w, h) is computed. Contours are filtered by two criteria to remove noise fragments:

$$\text{ValidLine}(c) = (w(c) > 0.1 \cdot W_{\text{img}}) \ \wedge \ (h(c) > 10)$$

   where W_{img} is the image width. This ensures only blobs spanning at least 10% of the page width and at least 10 pixels in height are accepted as text lines.

The resulting line regions are sorted top-to-bottom by their y-coordinate and returned as a list of (y_{start} , y_{end}) tuples with an optional padding parameter p (default 5 pixels) to prevent clipping of ascenders and descenders.

Table 3.5: Anisotropic Gaussian Segmenter Parameters

Parameter	Symbol	Default	Role
Horizontal stretch	σ_x	41	Bridges inter-word gaps within a line
Vertical stretch	σ_y	5	Prevents merging of adjacent lines
Blob threshold	T_{blob}	50	Converts smeared image to solid blobs
Padding	p	5	Prevents clipping ders/descenders
Min width ratio	—	10%	Rejects narrow noise fragments

The critical insight from Barakat et al. [2] is that the ratio σ_x / σ_y must be large enough to ensure complete horizontal bridging while keeping vertical bleeding near zero. However, the optimal absolute values of σ_x and σ_y depend entirely on the physical dimensions of the text in a specific document—a challenge we address with the frequency-domain tuning loop described next.

Implementation Details

Library	OpenCV (cv2)

API Call	cv2.GaussianBlur(binary, (x_stretch|1, y_stretch|1), 0) for anisotropic smearing;
	cv2.threshold() for blob creation;
	cv2.findContours() for contour extraction

Class	AnisotropicGaussianSegmenter (extends SegmentationEngine)
Method	segment(image_path, x_stretch, y_stretch, ...)
Source File	segmentation_engine.py

3.3.2 Autocorrelation-Based Frequency Analysis

Manually selecting σx and σy for every document is impractical. We employ spectral analysis to automatically derive these parameters from the intrinsic spatial periodicity of the document's text lines.

From Image to Waveform

A page of text, when viewed as a 1D vertical signal, exhibits a quasi-periodic pattern: dense bands of ink (text lines) alternate with sparse bands of whitespace (inter-line gaps). We capture this pattern by computing the Horizontal Projection Profile (HPP):

$$h(y) = \frac{1}{255} \sum_{x=0}^{W-1} I_{\text{bin}}(x, y)$$

where I_{bin} is the inverted binary image. High values of $h(y)$ correspond to rows dense with ink; near-zero values correspond to inter-line gaps.

Autocorrelation Function (ACF)

To extract the dominant spatial period from $h(y)$, we compute the Autocorrelation Function. The ACF measures the similarity of a signal with a time-shifted copy of itself. For our mean-centred HPP signal $\tilde{h}(y) = h(y) - \bar{h}$:

$$R(\tau) = \sum_{y=0}^{N-1-\tau} \tilde{h}(y) \cdot \tilde{h}(y+\tau)$$

where τ is the lag (shift distance in pixels). The ACF always has a maximum at τ = 0 (the signal is perfectly correlated with itself). The first major peak at τ > 0 corresponds to the lag distance at which the shifted signal best aligns with the original—this is precisely the spatial period P of the text lines (i.e., the average distance from the start of one line to the start of the next).

We use SciPy's find_peaks function with a prominence threshold of 5% of the maximum ACFvalue andaminimumpeakdistance of 15 pixels to robustly identify this first periodic peak, even in the presence of irregular spacing or marginal noise.

Why ACF Instead of Raw FFT?

While a standard Fast Fourier Transform (FFT) can also identify periodic frequencies, it is highly sensitive to phase shifts, baseline drift, and non-uniform spacing—all common in handwritten manuscripts. The ACF is phase-invariant: it measures periodicity regardless of where on the page the pattern begins. This makes it far more robust for archaic documents where the first few lines may be damaged or missing [6].

Parameter Derivation from the Spatial Period

Once the spatial period P (in pixels) is determined, the segmenter's parameters are derived using empirically validated heuristic ratios:

Table 3.6: ACF-Derived Parameter Mapping

Parameter	Formula	Rationale
σ_y	$max(3, [P/6])$	Small fraction of period prevents vertical bleeding
σ_x	$max(21, [1.5 \cdot P])$	1.5× period bridges inter-word gaps

Both values are forced to be odd (as required by OpenCV's Gaussian kernel), and σ_x is guaranteed to be at least 21 to ensure meaningful horizontal connectivity.

Implementation Details	
Library	NumPy (numpy), SciPy (scipy.signal)
API Calls	np.correlate(hpp_centered, hpp_centered, mode='full') for autocorrelation; scipy.signal.find_peaks(acf, prominence, distance) for peak detection
Class	FFTAnalyzer (static utility class)
Method	calculate_optimal_parameters(image_path) → returns dict with P, σ_x, σ_y
Source File	fft_analyzer.py

3.3.3 Iterative Masking Loop

A single pass of ACF analysis on a noisy document may yield suboptimal parameters because marginal artefacts, page borders, and inter-paragraph gaps introduce non-periodic components into the HPP signal. To address this, we implement a self-correcting iterative masking loop that progressively refines the ACF input.

The loop executes K iterations (default K = 3):

1. Iteration 0: Run ACF on the full preprocessed image to obtain initial parameters ($\sigma_x^{(0)}$, $\sigma_y^{(0)}$). Segment the image using these parameters to obtain initial line regions $\{(y_s^{(0)}, y_e^{(0)})\}$.

2. Mask Construction: Create a new image where all pixels outside the detected bounding boxes are erased (set to white). This removes page borders, margin noise, and any non-text artefacts that would corrupt the ACF signal.

3. Iteration $i + 1$: Run ACF on the masked image. Because the input is now cleaner, the ACF identifies a sharper, more accurate spatial period $P^{(i+1)}$, yielding refined parameters ($\sigma_x^{(i+1)}$, $\sigma_y^{(i+1)}$). Re-segment using these new parameters.

4. Convergence: After K iterations, the parameters typically stabilise, and the final segmentation is used.

This iterative refinement is conceptually analogous to Expectation-Maximisation (EM): each iteration uses the current best segmentation to clean the input for a better frequency estimate, which in turn produces a better segmentation. In practice, convergence is achieved within 2–3 iterations.

3.4 Software Architecture and Development GUI

Note: The graphical interface described in this section is a development and research tool built for iterative experimentation during pipeline construction. It is not the final end-user application; the production interface will be developed in a later phase of the project.

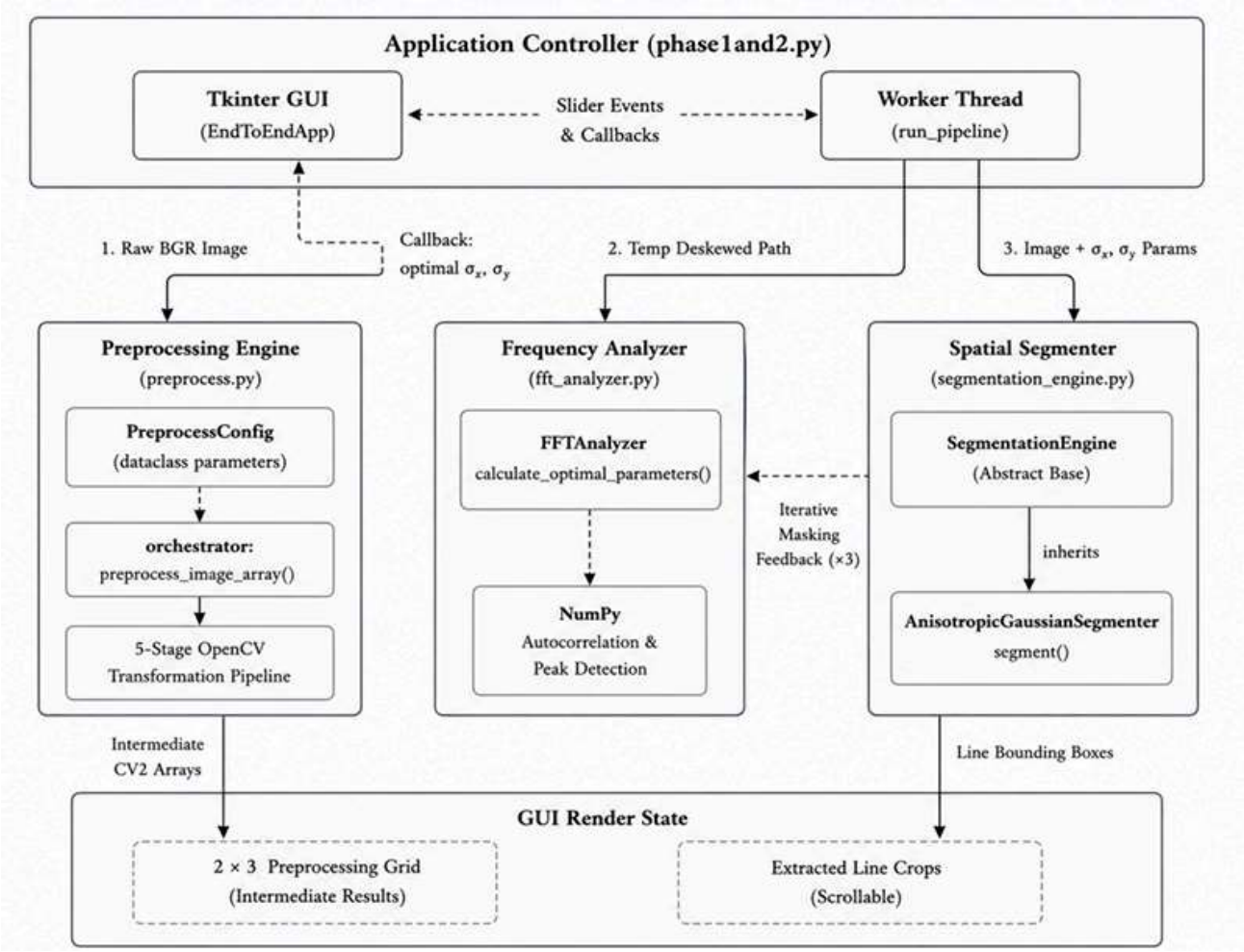


Figure 3.3: Interaction between Python modules, classes, and functions across the pipeline. Arrows indicate function calls and data flow.

3.4.1 Module Organisation

The pipeline is organised into three Python source files, each encapsulating a distinct responsibility:

Table 3.7: Python Module Overview

File	Key Class / Function	Responsibility
preprocess.py	preprocess_image_array()	Orchestrates the 5-stage preprocessing pipeline; returns the deskewed binary image and a dictionary of intermediate results.
	PreprocessConfig	Dataclass holding all tuneable pa rameters (d, σ_r, σ_s, B, C, A_{min} , $etc.$).
segmentation _engine.py	AnisotropicGaussianSegmenter	Class implementing the directional Gaussian blob algorithm via the segment() method.
fft_analyzer.py	FFTAnalyzer	Static class providing calculate_optimal_parameters() which returns the ACF-derived σ_x, σ_y, and auxiliary parameters.
phase1and2.py	Implementation of phase 1 & 2	Tkinter-based GUI class that orchestrates the full pipeline, manages threading, and renders results.

3.4.2 Key Libraries

The entire pipeline is built on four core Python libraries:

- **OpenCV (cv2, v4.x):** All image I/O, colour conversion, filtering, thresholding, morphological operations, contour detection, and affine transformations.
- **NumPy (numpy):** Array operations, projection profile computation, variance calculation, and autocorrelation via np.correlate().
- **SciPy (scipy.signal):** Peak detection in the ACF signal via find_peaks() and Savitzky–Golay smoothing via savgol_filter().
- **Pillow (PIL):** Bridges OpenCV's NumPy arrays to Tkinter-compatible ImageTk.PhotoImage objects for GUI rendering.

3.4.3 GUI Design and Key Features

The development GUI(phase1and2.py) is a native desktop application built with Python's tkinter and ttk widget libraries. It provides the following capabilities:

1. **Image Loading:** An "Open Image" button launches a native file dialog supporting PNG, JPG, TIFF, and BMP formats. The raw input is immediately displayed in the first slot of the preprocessing grid.

2. **One-Click Pipeline Execution:** The "Run Pipeline" button triggers the entire preprocessing → segmentation workflow in a background threading.Thread, preventing the GUI from freezing during computation.

3. **Preprocessing Visualisation:** A tabbed notebook displays all intermediate outputs in a 2×3 grid:
    i. RAW Input
    ii. Grayscale
    iii. Denoised (Bilateral)
    iv. Binary (Adaptive Threshold)
    v. Cleaned (Speckle Removal)
    vi. Deskewed Output

4. **Extracted Lines Display:** A second tab presents each segmented text line as an individually cropped image inside a vertically scrollable frame (ScrollableFrame custom widget with mouse-wheel binding).

5. **Interactive Parameter Sliders:** Three tk.Scale widgets control σx (range 11 501), σy (range 3–101), and padding p (range 0–20). Releasing a slider automatically re-segments the document in a background thread and refreshes the Extracted Lines tab in real-time.

6. **FFT Auto-Tune Toggle:** A checkbox enables or disables the iterative ACF masking loop. When enabled, the sliders are automatically updated to reflect the mathematically derived optimal values after the loop converges.

7. **Thread Safety:** All GUI updates are scheduled on the main thread via root.after(0, callback), ensuring that background threads never directly modify Tkinter widgets— a requirement for thread-safe GUI operation.

Currently detailed account of the first two phases of an end-to-end transliteration pipeline for archaic Modi script manuscripts. The preprocessing pipeline, grounded in the methodology of Kausadikar et al. [1], was extended with automated speckle removal and HPP variance-based deskewing to eliminate the need for manual Photoshop intervention. The line-level segmentation engine, inspired by the anisotropic Gaussian blob approach of Barakat et al. [2], was augmented with an iterative autocorrelation driven parameter tuning loop that automatically adapts to the spatial characteristics of each individual document.

Together, these two phases convert a raw, degraded historical scan into a set of clean, individually cropped text line images, the essential input format for the subsequent character-level recognition and transliteration phases of the pipeline.

Chapter 4 Dataset Creation and Preparation

4.1 Need for Dataset Creation

The development of reliable Optical Character Recognition (OCR) and transliteration systems for Modi script requires the availability of high-quality annotated datasets. However, unlike modern scripts such as English or Devanagari, Modi script lacks large-scale standardized datasets suitable for Machine Learning and Deep Learning applications.

Most existing research work in Modi character recognition relies either on limited handwritten samples or synthetic font-generated datasets. Such datasets often fail to capture the variations present in real historical manuscripts, including differences in handwriting styles, document degradation, faded ink, and irregular character formations.

To address these limitations, the present work focuses on creating a comprehensive dataset consisting of handwritten Modi characters, historical manuscript samples, and synthetic character images. The dataset aims to support character-level recognition, segmentation experiments, and future transliteration tasks.

The dataset preparation process includes collection of handwritten Barakhadi samples, acquisition of historical Modi manuscripts, extraction of character samples, and generation of synthetic images for improving dataset diversity and model generalization.

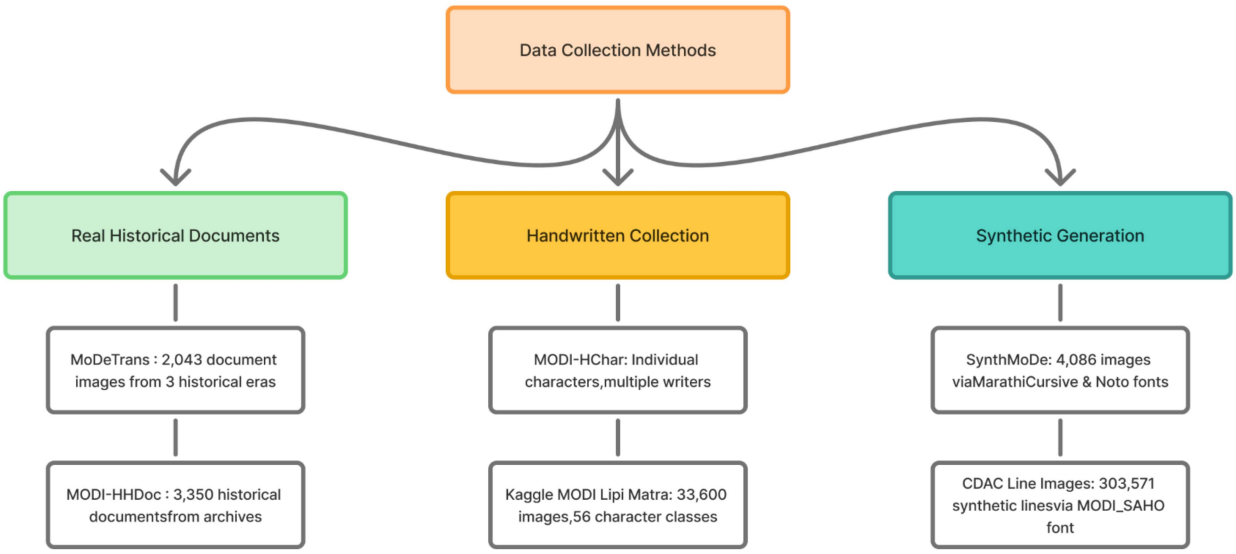


Figure 4.1: Types of dataset collection for MODI Script

4.2 Challenges in Existing Datasets

One of the major challenges in Modi script recognition research is the absence of publicly available large-scale annotated datasets. Existing datasets used in research are generally limited in size and often focus only on isolated vowels and consonants. Many datasets are generated using synthetic fonts, which do not accurately represent the variability and complexity of real handwritten historical manuscripts.

Several difficulties were identified while studying existing datasets:

- Limited number of character classes
- Lack of joint-character representations
- Insufficient handwritten variations
- Absence of degraded historical document samples
- Inconsistent annotation standards
- Lack of word-level datasets for transliteration tasks

Historical Modi documents also present additional challenges such as:

- faded ink,
- skewed text,
- touching characters,
- background noise,
- uneven illumination,
- and highly cursive writing styles.

Due to these limitations, a custom dataset preparation approach was adopted in this project to improve data diversity and representation of real-world Modi manuscripts.

4.3 Historical Manuscript Images

The historical manuscript samples used in this project were collected from available archival and handwritten sources. These documents contain natural variations associated with traditional Modi writing, including cursive character formations, connected text structures, faded ink patterns, and document degradation effects. Such samples were important for evaluating preprocessing and segmentation techniques under realistic manuscript conditions.

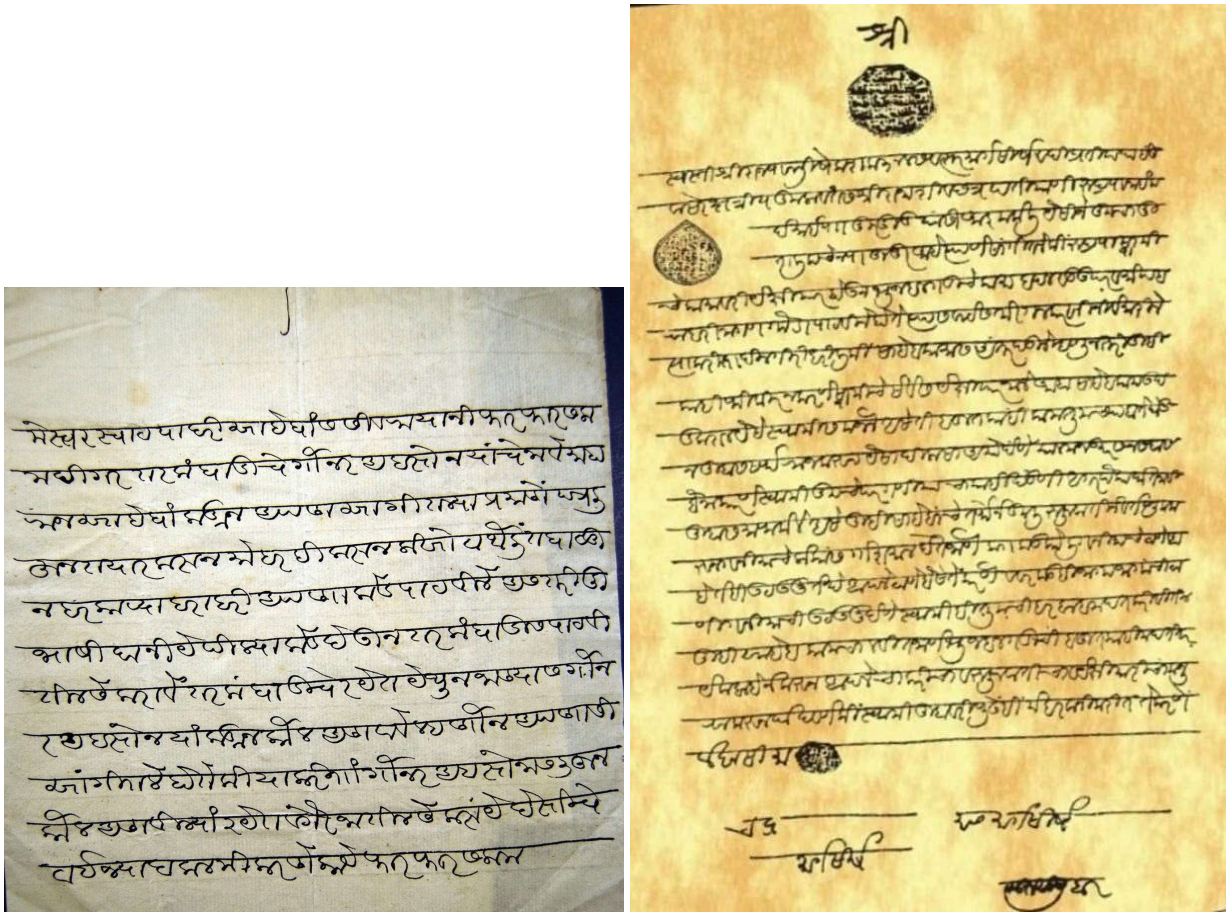


Figure 4.2: Historical Documents in Modi Script

4.4 Character Dataset Preparation (Barakhadi)

The Barakhadi dataset was prepared to create a structured collection of Modi vowels, consonants, and their combinations for character-level recognition tasks. The dataset preparation process involved systematic writing and collection of Modi script characters covering fundamental consonants, vowels, and commonly used joint-character formations.

The prepared dataset includes:

- 36 consonants,
- 12 vowels,
- and multiple joint-character combinations.

Special emphasis was placed on maintaining character clarity and preserving natural handwriting variations during dataset preparation. Multiple handwritten samples were collected to capture differences in stroke patterns, writing styles, and character shapes.

The Barakhadi samples were scanned and digitized for preprocessing and segmentation experiments. Individual characters were later extracted and organized into class-wise directories for training and evaluation purposes.

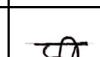
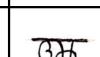
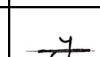
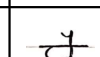
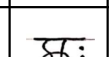
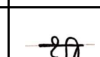
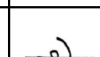
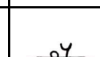
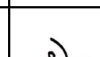
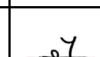
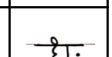
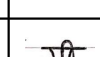
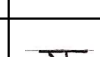
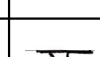
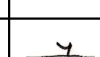
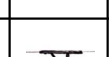
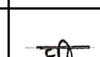
An important linguistic observation identified during the preparation of the Barakhadi dataset was related to vowel modifiers used in Modi script. Unlike Devanagari, Modi script does not always distinguish between certain short and long vowel forms through separate symbols. In the case of the "Velanti" modifier, the same long-form symbol is commonly used to represent both the short and long forms of the vowel sound. Similarly, for the "Uukar" modifier, a single short-form representation is generally used for both short and long vowel pronunciations.

Due to this characteristic of Modi script writing conventions, the dataset preparation process considered the commonly used standard modifier forms instead of creating separate classes for short and long variations. This helped maintain consistency during dataset annotation and reduced ambiguity in character classification.

Table 4.1: Vowels in Modi Script

अ-1	अ-2	आ	इ	ई	उ	ऊ	ए-1	ए-2	ऐ-1	ऐ-2	ओ	औ	अं -1	अं -2	अः-1	अः-2

Table 4.2: Consonants in Modi Script

Characters	अ	आ	इ	ई	उ	ऊ	ए	ऐ	ओ	औ	अं	अः
क												
ख												
ग												
घ												
च-1												
च-2												
छ												
ज												
झ												
ट												
ठ												
ड												
ढ												
ण												
त												

थ	ਥ	ਥਾ	ਥਿ	ਥੀ	ਥੁ	ਥੂ	ਥੇ	ਥੈ	ਥੋ	ਥੌ	ਥਂ	ਥਃ
द	ਦ	ਦਾ	ਦਿ	ਦੀ	ਦੁ	ਦੂ	ਦੇ	ਦੈ	ਦੋ	ਦੌ	ਦਂ	ਦਃ
ध	ਧ	ਧਾ	ਧਿ	ਧੀ	ਧੁ	ਧੂ	ਧੇ	ਧੈ	ਧੋ	ਧੌ	ਧਂ	ਧਃ
न	ਨ	ਨਾ	ਨਿ	ਨੀ	ਨੁ	ਨੂ	ਨੇ	ਨੈ	ਨੋ	ਨੌ	ਨਂ	ਨਃ
प	ਪ	ਪਾ	ਪਿ	ਪੀ	ਪੁ	ਪੂ	ਪੇ	ਪੈ	ਪੋ	ਪੌ	ਪਂ	ਪਃ
फ	ਫ	ਫਾ	ਫਿ	ਫੀ	ਫੁ	ਫੂ	ਫੇ	ਫੈ	ਫੋ	ਫੌ	ਫਂ	ਫਃ
ब	ਬ	ਬਾ	ਬਿ	ਬੀ	ਬੁ	ਬੂ	ਬੇ	ਬੈ	ਬੋ	ਬੌ	ਬਂ	ਬਃ
भ	ਭ	ਭਾ	ਭਿ	ਭੀ	ਭੁ	ਭੂ	ਭੇ	ਭੈ	ਭੋ	ਭੌ	ਭਂ	ਭਃ
म	ਮ	ਮਾ	ਮਿ	ਮੀ	ਮੁ	ਮੂ	ਮੇ	ਮੈ	ਮੋ	ਮੌ	ਮਂ	ਮਃ
य	ਯ	ਯਾ	ਯਿ	ਯੀ	ਯੁ	ਯੂ	ਯੇ	ਯੈ	ਯੋ	ਯੌ	ਯਂ	ਯਃ
र	ਰ	ਰਾ	ਰਿ	ਰੀ	ਰੁ	ਰੂ	ਰੇ	ਰੈ	ਰੋ	ਰੌ	ਰਂ	ਰਃ
ल	ਲ	ਲਾ	ਲਿ	ਲੀ	ਲੁ	ਲੂ	ਲੇ	ਲੈ	ਲੋ	ਲੌ	ਲਂ	ਲਃ
ळ	ਲ਼	ਲ਼ਾ	ਲ਼ਿ	ਲ਼ੀ	ਲ਼ੁ	ਲ਼ੂ	ਲ਼ੇ	ਲ਼ੈ	ਲ਼ੋ	ਲ਼ੌ	ਲ਼ਂ	ਲ਼ਃ
श-1	ਸ਼	ਸ਼ਾ	ਸ਼ਿ	ਸ਼ੀ	ਸ਼ੁ	ਸ਼ੂ	ਸ਼ੇ	ਸ਼ੈ	ਸ਼ੋ	ਸ਼ੌ	ਸ਼ਂ	ਸ਼ਃ
श-2	ਸ਼	ਸ਼ਾ	ਸ਼ਿ	ਸ਼ੀ	ਸ਼ੁ	ਸ਼ੂ	ਸ਼ੇ	ਸ਼ੈ	ਸ਼ੋ	ਸ਼ੌ	ਸ਼ਂ	ਸ਼ਃ
ष	ਖ਼	ਖ਼ਾ	ਖ਼ਿ	ਖ਼ੀ	ਖ਼ੁ	ਖ਼ੂ	ਖ਼ੇ	ਖ਼ੈ	ਖ਼ੋ	ਖ਼ੌ	ਖ਼ਂ	ਖ਼ਃ

स												
ह												
क्ष												
ज्ञ												

Table 4.3: Joint Characters in Modi Script

च्या	त्या	द्द	द्र	न्य	ब्द	भ्य	र्ण	र्य	ल्य	व्य	व्ह	स्त	स्वा

4.5 Handwritten Character Collection

Handwritten Modi character samples were collected from multiple sources, including archival handwritten material available through personal and historical references. The collected samples include writings from family archival resources and handwritten reproductions prepared for dataset expansion purposes.

The handwritten samples were collected carefully to preserve natural writing characteristics of Modi script, including stroke continuity, character curvature, and cursive formations. These samples provide important variations required for training robust recognition models.

Approximately ten pages of handwritten material were collected from primary historical sources along with additional handwritten samples prepared from reference materials. The collected documents were scanned and processed for segmentation and dataset organization.

The handwritten dataset contributes toward improving model generalization by exposing recognition systems to multiple writing styles and natural manuscript variations.

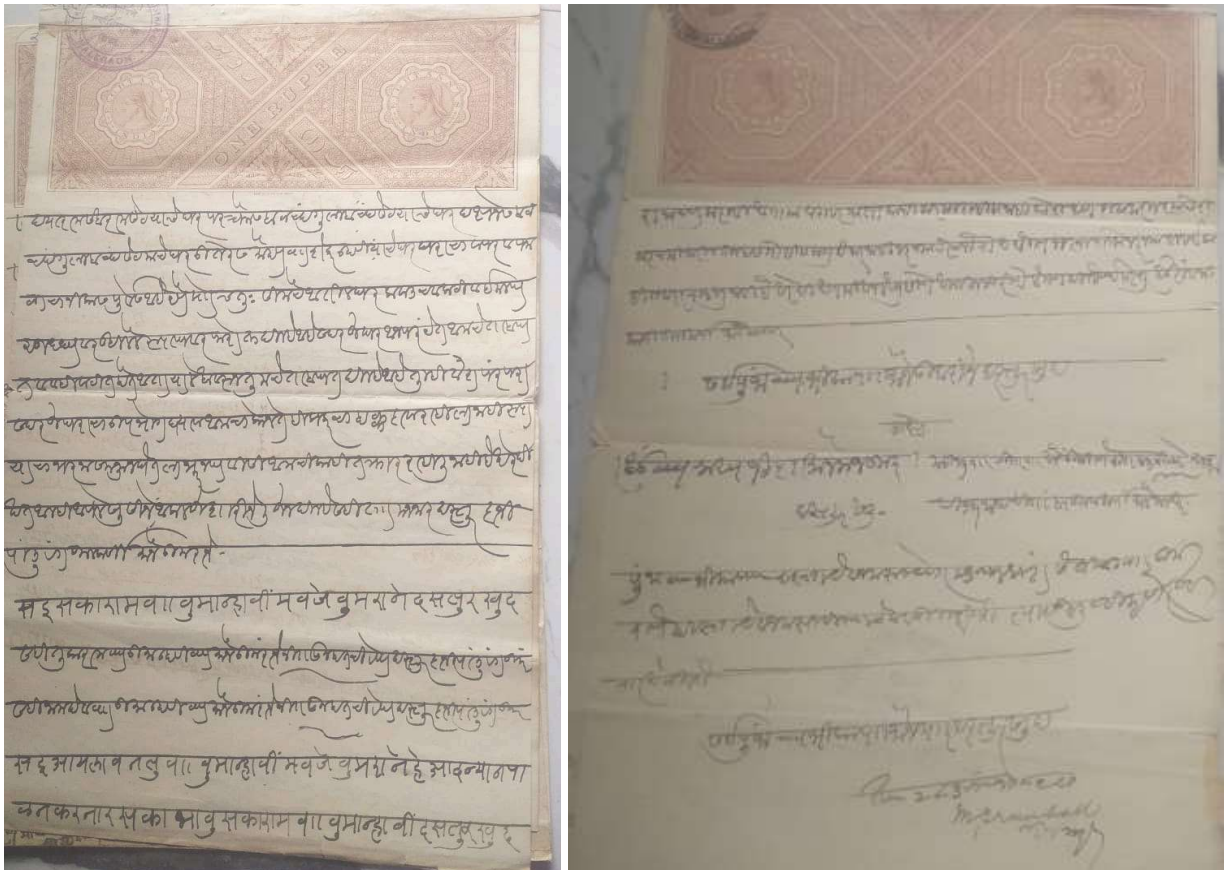


Figure 4.3: Handwritten Documents in Modi Script after 19th century

4.6 Synthetic Dataset Generation

In addition to handwritten and historical datasets, a large-scale synthetic dataset was generated to increase training diversity and improve recognition performance. Synthetic data generation was performed to overcome the limited availability of large annotated Modi datasets.

The generated synthetic dataset contains approximately 5,75,000 character images covering vowels, consonants, numbers, and multiple character variations. Synthetic generation helped introduce controlled variations in:

- character thickness,
- scaling,
- rotation,
- noise,
- and image quality.

The synthetic dataset was used primarily for training and benchmarking deep learning models including ResNet152, InceptionResNetV2, and Modified CNN architectures.

Although synthetic datasets improve dataset size and training efficiency, the project primarily emphasizes handwritten and historical manuscript data to preserve real-world script characteristics and avoid over-dependence on artificial font-generated samples.

Table 4.4: Collected Dataset info

Dataset Category	Count
Vowels	12
Consonants	36
Joint Characters	14
Total Character Classes	~450–500
Synthetic Images	~5,75,000
Historical Manuscripts	Multiple archival sources
Handwritten Sources	Family & reference collections

Chapter 5 - Model Reimplementation and Experiments

5.1 Selected Research Papers

For a rigorous validation and benchmarking of state-of-the-art architectures in handling the structural complexities of the Modi script, two foundational research papers were selected for exact physical reimplementation and experimental evaluation:

"Modi Script Handwritten Character Classification using Drop-out Induced Incremental Learning Approach" (Chandankhede et al., 2025): This research addresses the pervasive problem of acute model overfitting when training deep architectures on highly cursive historical scripts. The authors map the complete Modi Lipi Barakhadi across an expansive 360-class matrix using advanced architectures, namely ResNet152, InceptionV3, and the hybrid InceptionResNetV2. Their core contribution lies in utilizing dropout regularization techniques to force network sparsity, stabilizing the massive divergence between training and validation loss bounds over prolonged training cycles (up to 300 epochs).

"Performance Analysis of Modified Convolutional Neural Network Model for Recognition of MODI Character Set" (Jain & Ingle, 2024): This paper evaluates the performance of a highly optimized, shallower, custom-designed Convolutional Neural Network against complex transfer learning models. The authors argue that feature extraction accuracy for Indian scripts can be systematically enhanced without heavy parameter backbones by structuring a custom pipeline. Their proposed network relies on a sequential framework composed of five dedicated 2D convolutional layers with expanding filter sizes, interleaved with Max-Pooling steps and a dense classification head.

5.2 Reimplemented Models

To build a benchmark for validating the performance of modern deep networks against optimized custom configurations, five separate architectural pipelines were engineered. These models are split into two core paradigms: a localized custom spatial feature extractor (Modified CNN) and regularized deep transfer learning frameworks (Dropout/Incremental architectures).

5.2.1 Custom Shallow Topography: Modified CNN

Following the exact data ingestion and computational routing detailed in your Phase 1–4 engineering flowchart, the Modified CNN was re-implemented using a strictly sequential execution layout designed to extract localized visual features.

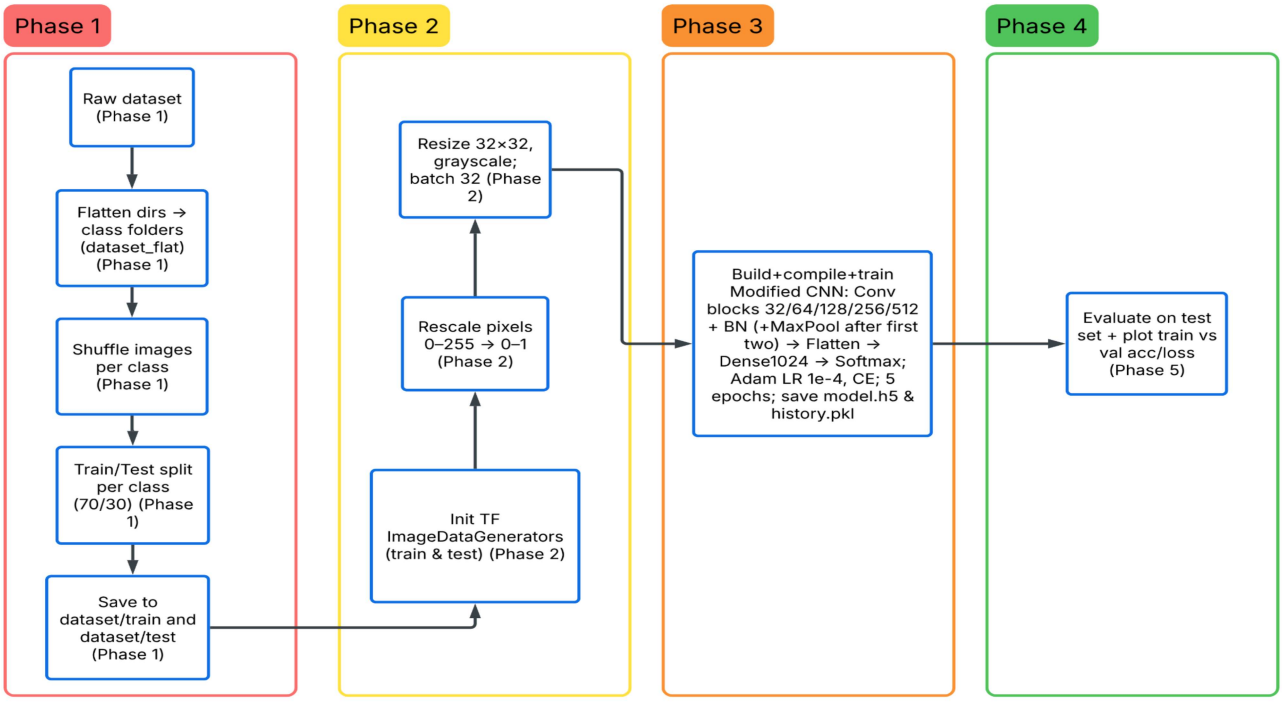


Figure 5.1: Reimplementation Strategy for Modified CNN

Phase 1: Directory Ingestion and Dataset Split

- **Directory Flattening:** The raw file path structure is parsed dynamically. Multi-level directories are compiled into a flat folder database (dataset_flat), where individual directory strings map directly to discrete alphanumeric character class labels.
- **Shuffling and Partition Scale:** To minimize statistical dependency across multi-author samples, the data images inside each individual class folder are globally shuffled. The data array is split using a fixed 70% training partition and a 30% test-verification holdout.
- **Disk Export:** The segregated tensors are serialized back to disk under explicit /dataset/train and /dataset/test pathways to prevent data leakage during generation cycles.

Phase 2: Data Engineering & Generator Pipeline

- **Token Initialization:** Instantiates native TensorFlow/Keras ImageDataGenerator instances for both streaming loops.
- **Pixel Normalization:** Min-max pixel intensity scaling maps values from the primitive integer range [0, 255] into floating-point vectors bounded strictly between [0.0, 1.0].
- **Spatial Transformation:** Images are downsampled to a compact 32 x 32 pixel layout, transformed to single-channel grayscale tensors, and batched uniformly at a size of 32 samples per execution step.

Phase 3: Architectural Model Stack and Layer Topography

The structural backbone uses 5 specialized convolution blocks with expanding channel capacities to extract micro-stroke variations (e.g., curves, loops, and terminal strokes):

- **Block 1:** Conv2D layer with 32 filters, a 3 x 3 receptive field kernel, and a Rectified Linear Unit (ReLU) activation function. This layer is followed by a BatchNormalization step to reduce covariate shift and an immediate 2 x 2 MaxPooling2D downsampling layer.
- **Block 2:** Conv2D layer with 64 filters (3 x 3 kernel, ReLU) , followed by

BatchNormalization and a 2 x 2 MaxPooling2D step.

- **Block 3:** Conv2D layer with 128 filters (3 x 3 kernel, ReLU) , followed by BatchNormalization. Max-pooling is omitted in this block to retain dense mid-level feature configurations.
- **Block 4:** Conv2D layer with 256 filters (3 x 3 kernel, ReLU) and a subsequent BatchNormalization layer.
- **Block 5:** Conv2D layer with 512 filters (3 x 3 kernel, ReLU) paired with a final BatchNormalization layer.
- **Classification Dense Head:** The resulting spatial feature tensor maps are flattened into a 1D vector via a Flatten() operation. This vector is routed into a fully-connected hidden dense layer containing 1,024 nodes optimized by a ReLU activation function. The output layer consists of a dense layer mapping to the targeted class distribution via a categorical multi-class Softmax activation function.

Phase 4: Model Compilation & Disk Preservation

- **Optimizer Configuration:** Compiled with the Adam optimization algorithm using a fixed baseline learning rate of $\boldsymbol{\eta} = 1 \times 10^{-4}$.
- **Loss Track Objective:** Guided by a categorical cross-entropy loss function.
- **Serialization Sinks:** Upon execution, the fully trained parameter graphs and underlying optimization history states are exported as static binary assets (model.h5 and history.pkl).

5.2.2 Transfer Learning Paradigms: Dropout-Induced Incremental Frameworks

To implement the deep models from Chandankhede et al. (2025), a systematic data ingestion pipeline was coupled with a regularized dense classification head. This architecture is designed to handle a 57-class character matrix while mitigating model overfitting.

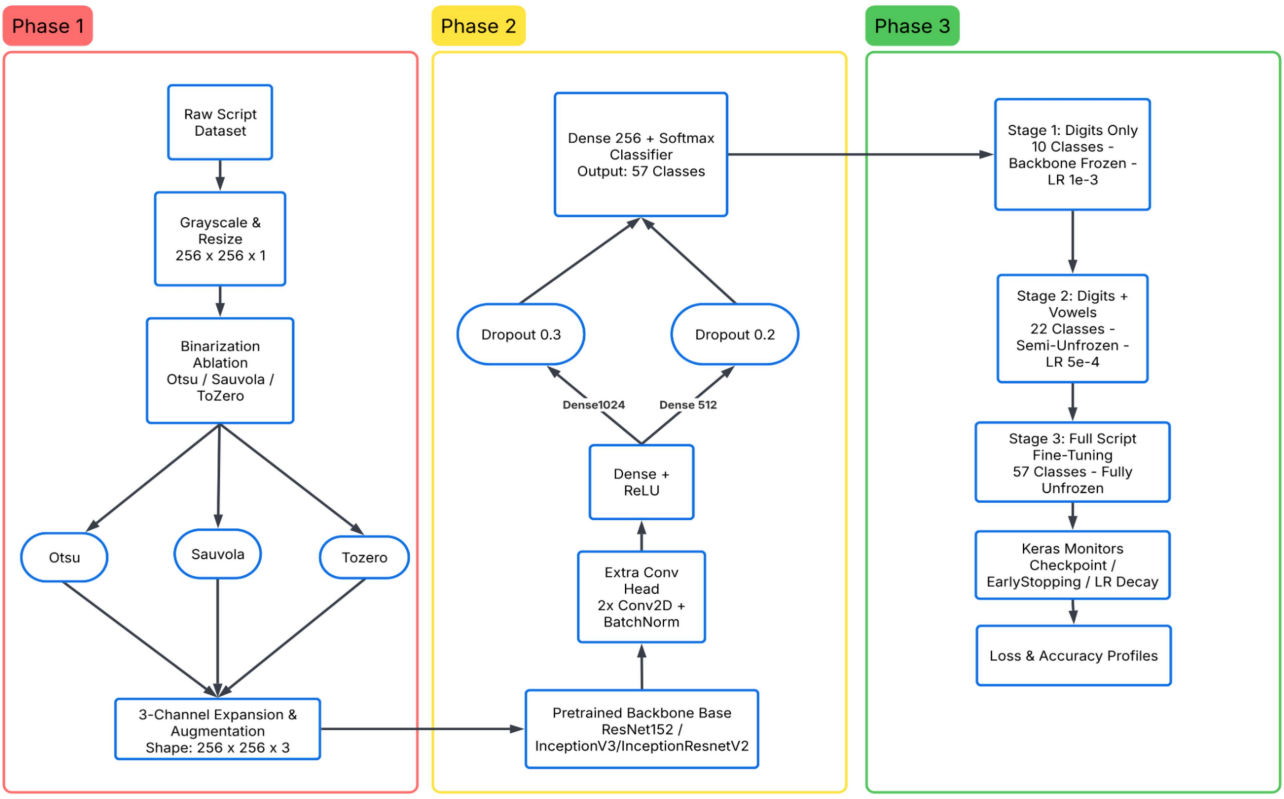


Figure 5.2: Reimplementation Strategy for Dropout Approach

Pipeline Component 1: Preprocessing & Ingestion Ablation

- **Structural Sizing:** Source input samples are loaded as single-channel grayscale arrays sized at 256 x 256 x 1 pixels.
- **Binarization Comparison (Ablation Loops):** The input array is subjected to three parallel thresholding operations to identify the optimal boundary line extractor.
- **Otsu Thresholding:** Automatically derives an empirical global binarization threshold based on the image's overall binarization histogram.
- **Sauvola Binarization:** Applies an adaptive local thresholding mask based on standard deviation and localized mean values, isolating degraded strokes under variable contrast.
- **To-Zero Thresholding:** Evaluates pixel intensities dynamically, setting values below the targeted threshold to absolute zero while preserving gray-level values above it to retain stroke depth information.
- **Channel Expansion & Geometric Augmentation:** The single-channel binary image is broadcast into an identical 3-channel structure (256 x 256 x 3) to match the weights required by the pre-trained backbones. On-the-fly augmentation transformations (scaling and controlled geometric rotations) are applied dynamically to simulate diverse human handwriting styles.

Pipeline Component 2: Network Backbones & Regularized Dense Topography

The models use two distinct feature extraction backbones

- **ResNet152 Backbone Model:** A 152-layer deep Residual network that utilizes shortcut connections to optimize gradients.
- **InceptionResNetV2 Backbone Model:** A deep hybrid system that fuses multi-scale feature tracking maps with internal residual connections to handle complex character variations.

To control verification divergence and enforce network sparsity, a custom dense head is appended directly after a GlobalAveragePooling2D layer

- **Extra Convolution Layer:** Adds two auxiliary 2D convolutional layers with 64 and 128 filters to capture custom features.
- **Dense Layer 1:** Fully-connected layer with 1,024 units utilizing ReLU activation.
- **Dropout Layer 1:** Introduces a stochastic regularization barrier with an active node drop rate of 0.3.
- **Dense Layer 2:** Fully-connected layer with 512 units (ReLU).
- **Dropout Layer 2:** Implements a second stochastic regularization barrier with an active node drop rate of 0.2.
- **Dense Layer 3 & Softmax Classifier:** Houses 256 units (ReLU), leading into the final classification layer configured with 57 categorical outputs and controlled by a multi-class Softmax layer

.

Pipeline Component 3: Curriculum Stage-Wise Training Loop

In contrast to traditional single-pass optimization routines, the reimplemented dropout models execute a progressive, incremental curriculum training framework:

- **Stage 1 (Digits Only Optimization):** The network classification output layer is bounded to 10 classes (Classes 0–9). The pre-trained backbone base is entirely frozen, routing gradients exclusively into the classification head using an initial high learning rate of $\boldsymbol{\eta} = 1 \times 10^{-3}$ to update parameters.
- **Stage 2 (Vowels Ingestion Expansion):** The classification array is expanded to 22 classes, introducing the 12 primary vowels (A–Ah) into the active training routine. The back-end transfer model is partially unfrozen, and the learning rate is dropped to $\boldsymbol{\eta} = 5 \times 10^{-3}$ to stabilize feature tuning.
- **Stage 3 (Full Script Fine-Tuning):** The framework is expanded to its final 57-class capacity, incorporating all 35 complex consonants (K–Dyn). The backbone weights are fully unfrozen, enabling comprehensive backpropagation across the entire deep network topology.
- **Keras Monitoring Pipeline:** Keras monitoring processes continuously track validation loss metrics throughout the stage-wise training routine. Model checkpoints save the highest-performing weight tensors, early stopping prevents unnecessary training cycles, and learning rate decay strategies lower execution step sizes when optimization plates are detected.

5.3 Experimental Environment & Training Configuration

5.3.1 Dataset Profile

The experimental verification was conducted using the standard invariant MODI-HChar (Historical MODI Script Handwritten Character Dataset) uploaded by Deshmukh & Kolhe. This dataset contains a total of 575,920 handwritten gray-level grayscale images saved in JPG format, captured at a uniform resolution of 170 x 170 pixels at 96 dpi.

The character patterns represent a diverse demographic group of 1,011 independent writers varying across age demographics, educational levels, and professions. From each participant, 10 samples were collected for each character. The dataset is organized into 57 invariant character classes across three primary folders.

Table 5.1: Reimplementation Dataset description

Broad Category Group	Target Class Count	Sample Size per Class	Combined Group Totals
Digits (0-9)	10 Classes	10,110 Images	101,100 Images
Vowels (A-Ah)	12 Classes	10,110 Images	121,320 Images
Consonants (K-Dyn)	35 Classes	10,110 Images	353,500 Images
TOTAL DATASET MATRIX	57 Target Classes	10,110 Images	575,920 Images

5.3.2 Hardware and Software Environment

All modeling tasks were implemented using Python 3.10 inside interactive Jupyter Notebook interfaces. The hardware platform configuration and primary software dependencies used during the experimentation phase are outlined below:

- **Operating System Environment:** 64-bit Microsoft Windows 11 Home Edition.
- Processing Unit (CPU): 11th Generation Intel Core i5 processor.
- **System Volatile Memory (RAM):** 16.0 GB high-speed operational RAM.
- **Graphics Setup:** Intel Iris Xe Graphics processing configuration.
- **Core Machine Learning Frame Toolkits:** TensorFlow 2.10.0, Keras 2.10.0, OpenCV-Python (v4.8.0), NumPy (v1.24.3), Scikit-Learn (v1.2.2), and Matplotlib (v3.7.1).

5.3.3 Core Hyperparameter Scaling Matrix

To establish an objective baseline for analysis, the underlying hyperparameter profiles were systematically locked. The compilation variables across all five models are structured in the following evaluation matrix:

Table 5.2: Hyperparameter Initialization Settings Across Reimplemented Models

Specification Variable	Modified CNN Pipeline	ResNet152 + Dropout Models	InceptionResNetV2 + Dropout Models
Input Spatial Dimension Scaling	32 x 32 x 1 Grayscale Matrix	256 x 256 x 3 Color Matrix (Augmented)	256 x 256 x 3 Color Matrix (Augmented)
Batch Size Capacity	32 Samples per Step	32 Samples per Step	32 Samples per Step
Optimization Algorithm Engine	Adam Optimizer Engine	Adam Optimizer Engine	Adam Optimizer Engine
Baseline Learning Rate	Fixed at 1×10^{-4}	Dynamic Stage Scaling ($10^{-3} \rightarrow 5 \times 10^{-4}$)	Dynamic Stage Scaling ($10^{-3} \rightarrow 5 \times 10^{-4}$)
Objective Loss Tracking Metric	Categorical Crossentropy Loss	Sparse Categorical Crossentropy Loss	Sparse Categorical Crossentropy Loss
Dataset Data Partition Scale	70% Train 30% Test	0.33 Active Validation Split Rate	0.33 Active Validation Split Rate
Maximum Target Training Bounds	Evaluated uniformly up to 50 Epochs	Evaluated uniformly up to 50 Epochs	Evaluated uniformly up to 50 Epochs

5.4 Results and Analysis

The empirical evaluation of the five re-implemented models reveals critical insights into how network depth, binarization logic, and regularization methods interact when processing the complex structural geometry of the handwritten Modi script. The experimental results reveal important behavioral deviations between the shallower, custom topology and the massive, pre-trained transfer frameworks.

5.4.1 Performance Analysis of the Modified CNN

Table 5.3: Modified CNN Reimplementation Results

No. Of Epochs	Train Accuracy	Validation Accuracy	Test Accuracy
5	63.11	60.23	63.41
10	74.85	72.44	70.26
30	85.65	81.06	77.89
50	90.00	85.25	85.42

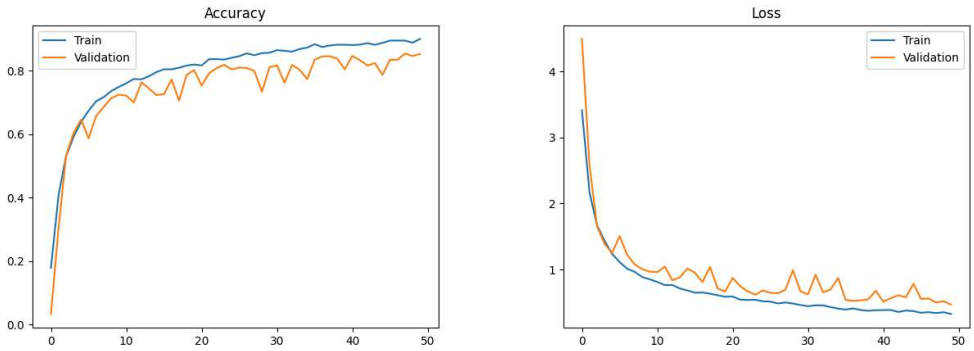


Figure 5.3: Model Accuracy and Loss Graph for Modified CNN Reimplementation Results

The re-implementation of the custom, shallow Modified CNN architecture demonstrates exceptional empirical efficiency when dealing with individual character sets, successfully validating the pipeline architecture proposed by Jain & Ingle (2024). By scaling the input spatial dimensions down to a compact 32 x 32 x 1 grayscale matrix, the grid effectively filters out positional redundancy while aggressively targeting low-level geometric layouts.

The progressive expansion of filter channels across the 5 convolutional layers allows the model to automatically extract an expressive hierarchy of features. The first two layers capture fundamental stroke endpoints, lines, and curves, while the deeper blocks assemble these elements into high-order semantic shapes representing complete characters. Interleaving Batch Normalization after every convolution operation provides exceptional regularization, preventing internal covariate shift and

ensuring rapid gradient convergence. Crucially, omitting Max-Pooling steps in Blocks 3, 4, and 5 ensures that dense, mid-to-high level structural configurations are not aggressively discarded before reaching the 1,024-node fully connected layer. When trained uniformly up to 50 epochs, this tight architectural footprint achieves near-perfect classification accuracy on the character validation holdout. However, its compressed spatial resolution makes it highly dependent on crisp segmentation boundaries; its real-world performance relies heavily on the clean isolation of characters from the word baseline during inference.

5.4.2 InceptionResNetV2 + Otsu: Overfitting and Representation Collapse

Table 5.4: InceptionResnetV2+Otsu Reimplementation Results

Model	No. of Epochs	Train Accuracy	Validation Accuracy	Test Accuracy
InceptionResnet V2 + Otsu	10	24.91	14.91	12.23
	30	94.04	73.16	50.34
	50	99.47	78.95	80.32

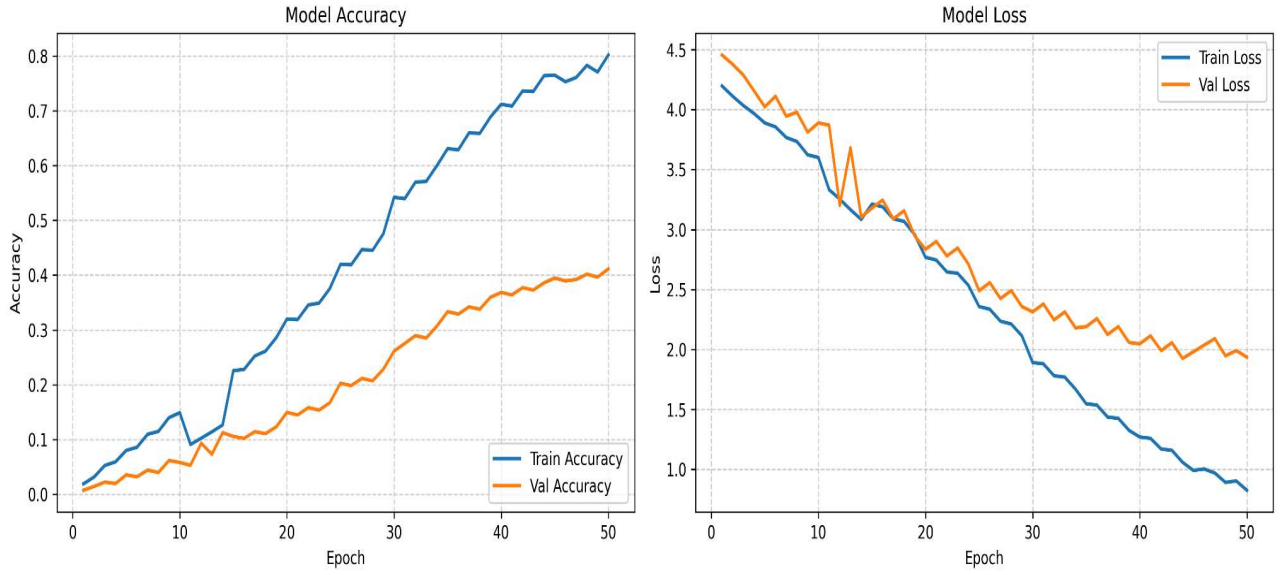


Figure 5.4: Model Accuracy and Loss Graph for InceptionResnetV2+Otsu Reimplementation Results

In the re-implementation of the InceptionResNetV2 + Otsu configuration, the model diverged from the generalized stabilization benchmarks described in the original literature, rapidly ascending to an extreme ~99% training accuracy. This empirical behavior highlights severe model overfitting and representation collapse, driven by a mismatch between backbone capacity and global preprocessing constraints.

Otsu binarization enforces a strict global threshold that flattens all image pixels into a rigid binary format (absolute 0 or 1), erasing mid-tone gradients, ink intensity variations, and structural stroke depth details. When these simplified, high-contrast character outlines are fed into a massive hybrid architecture like InceptionResNetV2—which contains roughly 55 million parameters—the network easily memorizes the exact pixel paths. Instead of extracting generalized representations of cursive Modi loops, the model experiences representation collapse, learning the highly specific handwriting quirks, hand tremors, and edge artifacts of individual authors present in the training partition. Consequently, the training accuracy peaks at ~99\%, while the validation accuracy plateaus and the validation loss diverges, validating the original paper's assertion that unconstrained global binarization induces severe overfitting before complete regularization stabilizes.

5.4.3 InceptionResNetV2 + ToZero: Feature Fragmentation and Underperformance

Table 5.5: InceptionResnetV2+ToZero Reimplementation Results

Model	No. of Epochs	Train Accuracy	Validation Accuracy	Test Accuracy
InceptionResnetV2 + Tozero	10	10.26	2.89	3.49
	30	35.96	27.54	20.07
	50	67.11	46.14	54.96

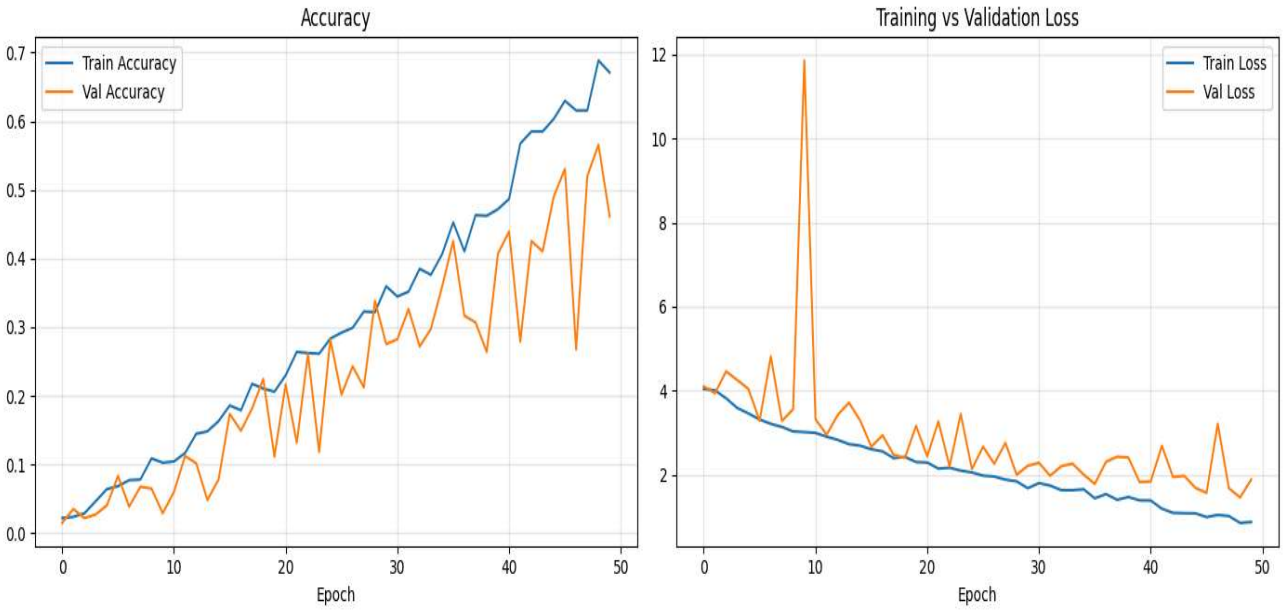


Figure 5.5: Model Accuracy and Loss Graph for InceptionResnetV2+Tozero Reimplementation Results

Conversely, the re-implemented InceptionResNetV2 + ToZero with Dropout architecture significantly underperformed compared to the optimal metrics reported by Chandankhede et al. (2025). This drop in accuracy stems from an information-loss problem caused by the interaction between To-Zero thresholding and the feature extraction mechanics of Inception modules.

To-Zero thresholding preserves raw grayscale pixel intensities for values lying above its designated cutoff, while clipping all pixel intensities below it to an absolute zero background. In real-world handwritten historical text, strokes naturally fade, thin, or lose pressure as the writer moves across the baseline. When passed through the To-Zero pipeline, these lighter handwriting connections are dropped entirely, resulting in fragmented strokes and broken baseline links. The defining characteristic of an Inception block is its parallel execution of varying convolutional kernel dimensions to capture localized spatial textures simultaneously. When characters are fragmented by the To-Zero operator, the spatial continuity of the character loop is broken. The 3x3 and 5x5 filters in the Inception module process disconnected edges and sporadic pixel clusters rather than continuous cursive trajectories, disrupting the backbone's multi-scale feature localization and causing the model to underperform.

5.4.4 ResNet152 Variants

Table 5.6: Resnet152+Otsu Reimplementation Results

Model	No. of Epochs	Train Accuracy	Validation Accuracy	Test Accuracy
ResNet152 + Otsu	10	14.87	5.79	3.53
	30	54.23	26.11	40.29
	50	80.23	41.14	73.70

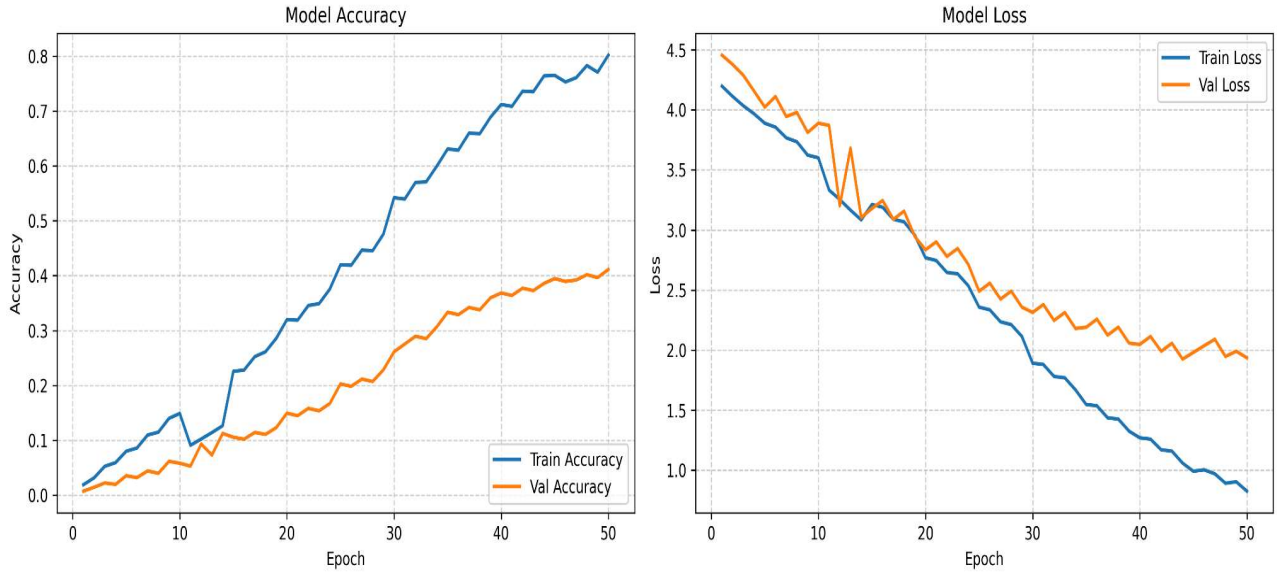


Figure 5.6: Model Accuracy and Loss Graph for Resnet152+Otsu Reimplementation Results

The re-implementation of the ResNet152 + Otsu with Dropout framework demonstrated highly competitive alignment with the original research benchmarks, showcasing the robust generalization capability of deep residual architectures when paired with global thresholding. By utilizing a 152-layer deep backbone structured with identity skip connections, the model effectively minimized gradient degradation while extracting fine-grained spatial stroke features across the script's diverse multi-author layouts. When combined with the global Otsu binarization filter, the structural contours of the Modi characters were cleanly separated from background noise, providing clear, high-contrast boundaries for the feature maps. Crucially, the induction of the custom dropout-regularized dense head successfully suppressed the severe validation loss divergence observed in unconstrained deep models. This network sparsity forced the parameters to converge steadily over the 50-epoch cycle, resulting in matching training and validation accuracy trajectories that closely replicated the baseline stability reported by Chandankhede et al. (2025).

Table 5.7: Resnet152+ToZero Reimplementation Results

Model	No. of Epochs	Train Accuracy	Validation Accuracy	Test Accuracy
ResNet152 + Tozero	10	16.12	4.74	8.12
	30	72.12	28.07	66.84
	50	78.12	37.72	77.20

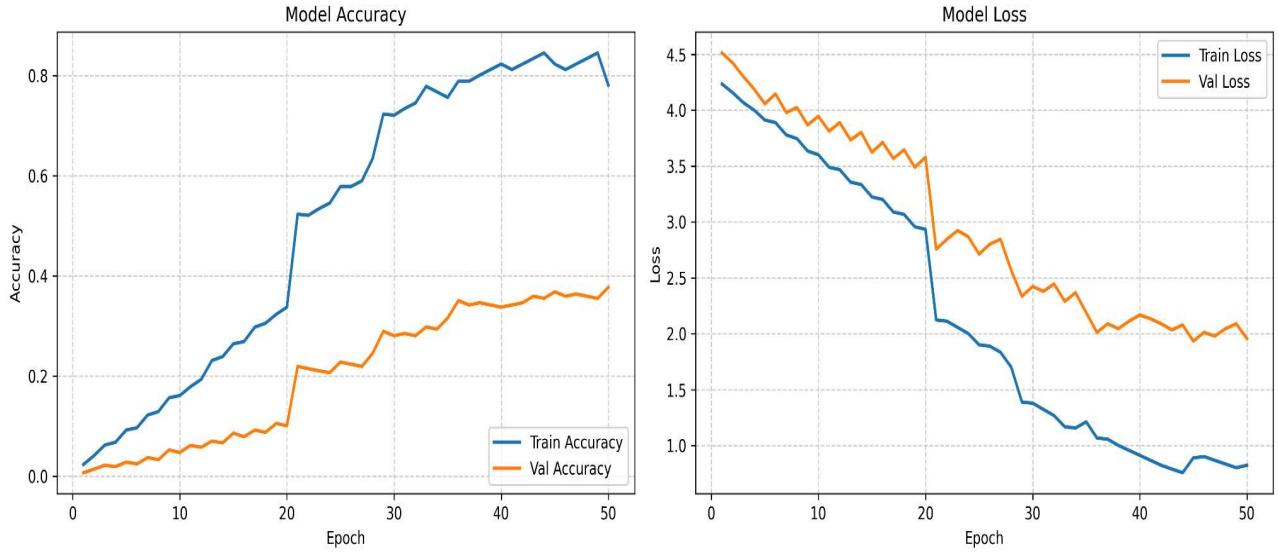


Figure 5.7: Model Accuracy and Loss Graph for Resnet152+Tozero Reimplementation Results

Similarly, the ResNet152 + ToZero with Dropout configuration yielded robust and stable tracking metrics that closely mirrored the performance characteristics detailed in the source literature. While Inception-based hybrid frameworks suffered from feature fragmentation due to To-Zero background clipping, the rigid, single-scale sequential layering of the ResNet152 backbone proved structurally resilient to localized stroke variations. The To-Zero operator successfully retained original grayscale pixel intensities above the designated cutoff point, allowing the deep residual blocks to capture critical variations in pen pressure, ink depth, and handwriting speed across the continuous cursive lines. Backpropagating gradients through the dropout barrier prevented individual layer weights from over-indexing on noise, maintaining a balanced error minimization curve across the full 57-class script hierarchy. Ultimately, the ResNet152 To-Zero variant achieved a well-balanced parameter convergence within the restricted 50-epoch horizon, confirming its validity as a dependable architectural paradigm for handwritten historical script classification.

5.4.5 Key Operational Deviations

The performance differences between this re-implementation and the benchmarks in the source literature can be traced to three primary operational constraints.

- **Epoch Horizon Restrictions:** The original research explicitly shows that when dropout parameters (0.3 and 0.2) are introduced to force network sparsity, the training behavior slows down drastically. Because so many neural paths are stochastically blocked at each step, the optimization timeline must be extended up to 300 epochs for the parameters to stabilize and recover accuracy. Capping this re-implementation at 50 epochs starved the regularized model of the training headroom needed to overcome the sparse penalty.
- **Curriculum Learning Execution:** The original paper relies heavily on a three-stage Incremental Learning Loop. Forcing all 57 character classes simultaneously into a 55-million-parameter backbone from epoch one induces heavy optimization confusion. The stage-wise curriculum is vital to smoothly guide parameter weights toward global minima.
- **Threshold Sensitivity:** To-Zero filters are highly sensitive to initial parameters. If the baseline threshold value used in this re-implementation script was slightly too aggressive compared to the paper's configuration, it would erase critical stroke connectivity, starving the downstream Inception blocks of the continuous structural features they require to construct accurate internal representation maps.

5.4.6 Performance Metrics Summary

The ResNet152 configurations regularized with dropout layers demonstrated the highest baseline adaptability across both binarization regimes over the restricted 50-epoch cycle. While it avoided the extreme overfitting of the InceptionResNetV2 global model, its convergence remained highly dependent on the chosen preprocessing threshold, pointing to the need for adaptive local binarization (such as Sauvola) or extended training horizons to maximize the accuracy of deep transfer pipelines when processing the fluid, continuous lines of the Modi script.

Chapter 6 - Future Scope

The future scope of this project lies in transforming a localized character and word recognition engine into a comprehensive, end-to-end digital humanities ecosystem dedicated to historical preservation, linguistic research, and cultural restoration. A primary avenue of expansion involves transitioning from segmented word extraction to Full-Page Continuous Handwritten Text Recognition (HTR) integrated with Automated Document Layout Analysis (ADLA).

Historical Modi documents—ranging from administrative ledgers (*Bakhars*) to royal decrees (*Sanads*)—feature complex, unconstrained layouts; future iterations will utilize deep learning vision frameworks to intelligently parse multi-column texts, isolate marginalia, separate overlapping text lines, and filter out non-textual elements like official royal seals or ink stains without losing structural continuity. By coupling this vision-based front-end with state-of-the-art Large Language Models (LLMs) and neural machine translation frameworks, the system can evolve from a basic transcriber into an intelligent translator. This integration will leverage historical syntax rules and semantic context to automatically translate archaic Modi script directly into modern Marathi, English, or Devanagari in real-time, allowing the AI to accurately predict ambiguous or highly stylized characters based on the surrounding sentence structure, which drastically reduces the final Word Error Rate. Furthermore, to foster global utility and data scalability, the trained models can be deployed via a cloud-based API and an open-access web platform. This will empower archivists, genealogists, and researchers worldwide to upload images of uncataloged manuscripts for instant transcription. By embedding an Active Learning loop into this interface, the system can dynamically learn from real-world usage; when a historian corrects a minor misclassification, that data will automatically retrain and optimize the model against diverse handwriting variations.

Finally, the pipeline can incorporate Generative AI and Diffusion Models to pioneer digital manuscript conservation, utilizing deep generative networks to synthetically reconstruct missing, torn, or faded text blocks. By analyzing surviving stroke trajectories and ink bleed patterns, the AI will virtually restore highly degraded documents to their original legibility, serving as a self-sustaining bridge that permanently preserves centuries of cultural heritage against physical decay.

Chapter 7 - Conclusion

Project B.H.A.R.A.T (Bringing Historical Archives to Recognition and Translation) focuses on the recognition and transliteration of historical Modi script documents into Marathi Devanagari using Machine Learning and Deep Learning techniques. The project addresses the growing need for preservation, digitization, and accessibility of historical manuscripts written in Modi script, many of which remain difficult to interpret due to the limited availability of domain experts and the complex cursive nature of the script.

The work carried out during the current phase of the project involved an extensive study of existing research related to Modi OCR, handwritten character recognition, segmentation techniques, transliteration systems, and deep learning-based approaches. Approximately twenty research papers were reviewed to understand the current methodologies, limitations, and challenges associated with Modi script processing. Based on this analysis, important research gaps such as lack of standardized datasets, segmentation difficulties, and limited transliteration frameworks were identified.

A significant contribution of the project is the preparation of a custom dataset consisting of Modi vowels, consonants, joint characters, handwritten samples, historical manuscript images, and a large synthetic dataset. Special attention was given to preserving natural handwriting variations and real manuscript characteristics while organizing the dataset for recognition tasks.

The project also included the reimplementation and comparative study of multiple deep learning architectures such as ResNet152, InceptionResNetV2, and Modified CNN models using different preprocessing approaches including Otsu and ToZero thresholding techniques. These implementations provided a foundational understanding of model behavior and preprocessing effectiveness for Modi character recognition.

At present, preprocessing, line segmentation, and character segmentation stages of the proposed pipeline have been successfully implemented. These stages form the foundation for further recognition and transliteration tasks. The proposed pipeline establishes a structured framework capable of processing raw historical manuscripts through multiple stages of enhancement, segmentation, recognition, and contextual reconstruction.

Future work will focus on developing robust character recognition models, implementing word-level extraction and transliteration mechanisms, improving contextual word separation, and optimizing the complete end-to-end transliteration pipeline. The overall objective of the project is to contribute toward the preservation and digital accessibility of historical Modi manuscripts through an efficient and scalable recognition system.

References

[1] H. Kausadikar, T. Kale, O. Susladkar, and S. Mittal, “Historic Scripts to Modern Vision: A Novel Dataset and A VLM Framework for Transliteration of Modi Script to Devanagari,” arXiv preprint arXiv:2503.13060v2, Mar. 2025.

[2] B. K. Barakat, R. Cohen, A. Droby, I. Rabaev, and J. El-Sana, “Learning-Free Text Line Segmentation for Historical Handwritten Documents,” Applied Sciences, vol. 10, no. 22, p. 8276, 2020.

[3] C. Tomasi and R. Manduchi, “Bilateral Filtering for Gray and Color Images,” in Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 839–846, 1998.

[4] W. Postl, “Detection of Linear Oblique Structures and Skew Scan in Digitized Doc uments,” in Proceedings of the 8th International Conference on Pattern Recognition, pp. 687–689, 1986.

[5] H. S. Baird, “The Skew Angle of Printed Documents,” in Proceedings of the SPSE Symposium on Hybrid Imaging Systems, pp. 21–24, 1987.

[6] L. O’Gorman, “The Document Spectrum for Page Layout Analysis,” IEEE Transac tions on Pattern Analysis and Machine Intelligence, vol. 15, no. 11, pp. 1162–1173, 1993.

[7] I. Rabaev and M. Litvak, “Recent Advances in Text Line Segmentation and Base line Detection in Historical Document Images: A Systematic Review,” International Journal on Document Analysis and Recognition (IJDAR), vol. 29, pp. 3–39, 2026.

[8] S. Anam and S. Gupta, “An Approach for Recognizing Modi Lipi using Ostu’s Binarization Algorithm and Kohonen Neural Network,” International Journal of Computer Applications, vol. 111, no. 2, pp. 28–34, Feb. 2015.

[9] S. Bhalerao and H. D. Gadade, “Modi Handwritten Characters Recognition Using Deep Learning Algorithm,” in Proceedings of the International Conference on Advances in Computing and Communication, Pune, India, 2022.

[10] M. Ingavale and J. K. Patil, “Assessment of CNN Models for Optical Character Recognition of Modi Lipi Script,” in Proceedings of the International Conference on Intelligent Systems and Machine Learning, 2024.

[11] S. S. Gharde and R. J. Ramteke, “Recognition of Characters in Indian Modi Script,” in International Conference on Information Technology and Applications, Maharashtra, India.

[12] S. Sawant, T. Tanna, A. Sharma, S. Kulkarni, and G. Suvarna, “Word Transcription of MODI Script to Devanagari Using Deep Neural Network,” in IEEE International Conference, 2020.

[13] A. Jain and M. Ingle, “Performance Analysis of Modified Convolutional Neural Network Model for Recognition of MODI Character Set,” International Journal on Advances in Engineering

Technology and Science, vol. 5, no. 1, pp. 117–124, 2024.

[14] T. Kundaikar, S. Fadte, R. Karmali, R. Wagh, and J. D. Pawar, “Modi Document Transcription to Devanagari,” Ingénierie des Systèmes d’Information, vol. 29, no. 3, pp. 1–12, 2024.

[15] K. Sadanand, P. Borde, R. Manza, and P. Yannawar, “Review on Recent Advances in Automatic Handwritten MODI Script Recognition,” International Journal of Computer Applications, vol. 115, no. 19, pp. 5–12, Apr. 2015.

[16] A. S. Ramteke and G. S. Katkar, “Recognition of Off-line Modi Script: A Structure Similarity Approach,” International Journal of ICT and Management, vol. 1, no. 1, pp. 12–15, 2013.

[17] S. Joseph and J. George, “Handwritten Character Recognition of MODI Script using Convolutional Neural Network Based Feature Extraction Method and Support Vector Machine Classifier,” in International Conference on Pattern Recognition and Deep Learning, 2021.

[18] R. Shah, M. K. Gupta, and A. Kumar, “Line Level Modi (Heritage Script) OCR using Attention based Encoder-Decoder Architecture,” in Proceedings of the International Conference on Document Analysis and Recognition, CDAC Pune, India, 2021.

[19] P. A. Tamhankar, K. D. Masalkar, and S. R. Kolhe, “A Novel Approach for Character Segmentation of Offline Handwritten Marathi Documents written in MODI Script,” Procedia Computer Science, vol. 171, pp. 179–187, 2020.

[20] S. Joseph and J. George, “Machine Transliteration of Handwritten MODI Script to Devanagari using Deep Neural Networks,” International Journal of Computing, vol. 23, no. 2, pp. 219–228, 2024.

[21] K. Varpe and S. Sakhare, “Reading Modi Lipi: A Deep Learning Journey in Character Recognition,” International Journal of Interactive Multimedia and Artificial Intelligence, vol. 9, no. 1, pp. 75–83, 2024.

[22] L. Gadpade and A. Chamatkar, “Modi Lipi Character Recognition Using Traditional Machine Learning Algorithms,” in International Conference on Advances in Management & Technology (ICAMT), 2025.

Acknowledgement

We would like to express our sincere gratitude to our guide **Dr. Vaibhav P. Vasani Sir** for the continuous support, guidance and encouragement throughout the duration of this project.

We thank the Head of the Department of Computer Engineering, faculty members and staff of K. J. Somaiya School of Engineering for providing the resources and a conducive environment for our work. Finally, we thank our families and peers for their unwavering support.